

Approcci 2D e 3D per l'analisi e la modellazione digitale di capi d'abbigliamento

Università di Verona

Rosa Russo

Settembre 2025

Contents

1	Fase di studio e analisi del lavoro esistente	3
1.1	Introduzione	3
1.2	Approcci 2D	3
1.2.1	End-to-end Recovery of Human Shape and Pose	3
1.2.2	Moulding Humans	4
1.2.3	BodyNet	4
1.2.4	Parsing clothing in fashion photograph	4
1.2.5	Multi-Garment Net	5
1.3	Approcci 3D	8
1.3.1	SPSD	8
1.3.2	BUFF	8
1.3.3	DeePSD	9
1.3.4	Sizer	12
1.3.5	SMPL	12
1.3.6	CAPE	15
1.3.7	ClothCap	19
2	Obiettivi	23
3	SIZER: dataset e modello per l'analisi e l'apprendimento dell'abbigliamento 3D size sensitive	23
4	Dataset	24
4.1	Confronto tra SIZER e altri datasets	25
5	Metodo	27
5.1	ParserNet	28
5.1.1	Rete per la predizione di posa e forma	28
5.2	Analisi dei vestiti	28
5.3	Analisi della body shape svestita	29
5.4	Funzione di Loss	29
5.4.1	3D Vertex Loss per vestiti	29
5.4.2	3D Vertex Loss corpi svestiti	30
5.4.3	Normal Loss	30
5.4.4	Termine di smoothness Laplaciano	30

5.4.5	Interpenetration loss	30
5.4.6	Regolarizzazione dei pesi	31
5.5	Cenno sulle reti neurali utilizzate	31
6	Codice	31
7	Linux	31
7.0.1	Ptbody-mesh	32
7.0.2	Kaolin	33
7.0.3	CUDA	34
7.0.4	Installazione di Kaolin	34
7.1	Download del dataset	36
8	Windows	37
8.1	Boost	37
8.2	Librerie Boost per GCC	37
8.3	Ptbody-mesh	37
8.4	Kaolin	38
8.5	Sizer dataset	38
8.6	Installazione dei pacchetti	38

1 Fase di studio e analisi del lavoro esistente

1.1 Introduzione

In questo paragrafo viene spiegato brevemente il percorso di analisi del lavoro esistente riguardo la modellazione virtuale di vestiti su mesh tre-dimensionali.

Mesh Una mesh poligonale è la partizione di una superficie continua in celle poligonali, come triangoli, quadrilateri, ecc. Più formalmente, una mesh M può essere definita come una *tupla* (V, K) , dove

$$V = \{\mathbf{v}_i \in \mathbb{R}^3 \mid i = 1, \dots, N_v\}$$

è l'insieme dei vertici del modello (punti in $\mathbb{R}^3$) e K contiene l'*adjacency information*, ovvero come i vertici sono connessi per formare gli spigoli e le facce della mesh.

Review Inizialmente sono stati studiati articoli che riguardano l'estrazione di mesh del corpo umano partendo da immagini e video, ma per seguire questa strada c'è bisogno prima di stimare la forma del corpo del soggetto in esame e poi procedere all'estrazione delle mesh dei singoli vestiti separate da quella del corpo stesso. Per questa ragione si è passato poi alla ricerca di dataset contenenti scansioni 3D ed articoli di lavori correlati riguardanti la stratificazione di mesh inerenti a corpo umano nudo ed indumenti indossati.

Tra i lavori ritenuti interessanti tra quelli suggeriti dal tutor, troviamo un self-supervised framework (SPICE) [10] che, partendo dall'immagine di una persona, è in grado di generare una nuova immagine di tale soggetto in una posa target, raggiungendo performance dello stato dell'arte sul DeepFashion dataset, con prestazioni migliori rispetto a metodi non supervisionati e simili allo stato dell'arte di metodi supervisionati. SPICE, inoltre, genera video partendo da una singola immagine data in input ed una sequenza di pose, nonostante sia stato addestrato solo su immagini statiche.

1.2 Approcci 2D

Volendo partire da immagini 2D per predire il volume del corpo umano svestito e la mesh dei vestiti indossati, sono stati analizzati i seguenti articoli:

- End-to-end Recovery of Human Shape and Pose [5],
- Moulding Humans [3],
- BodyNet [12],
- Parsing clothing in fashion photographs [14],
- Multi-garment Net [2].

1.2.1 End-to-end Recovery of Human Shape and Pose

In "End-to-end Recovery of Human Shape and Pose" [5] viene fatta una Human Mesh Recovery (HMR), viene ricostruita cioè una mesh 3D di un corpo umano, partendo dalle immagini RGB del dataset Human3.6M ¹. Da una GAN vengono generati parametri relativi a posa, shape e camera. Questo sembrerebbe estrarre meglio il corpo senza vestiti rispetto a BodyNet e Moulding humans, ma non è sempre assicurato che la forma corrisponda alla realtà (ad esempio, per quanto riguarda foto di donne incinta, nella costruzione della forma 3D viene eliminata la pancia perché

¹<http://vision.imar.ro/human3.6m/description.php>

confusa con gli abiti). Questo perché la rete si occupa principalmente di generare una shape che sembri reale, piuttosto che ottenerne una corrispondente al corpo originale ritratto in foto. Dunque necessiterebbe di migliorie quali l'aggiunta di tipi di forme del corpo non previste nel lavoro originale. Il dataset Human3.6M, mette a disposizione 3.6 milioni di immagini di persone e le corrispondenti pose e mesh, acquisite mediante quattro camere digitali. I dati sono organizzati in quindici movimenti di training, tra i quali camminare con vari tipi di asimmetrie (per esempio, camminare con una mano nella tasca, o con una borsa sulla spalla), pose di persone in attesa, sedute o sdraiate, ecc. Tali movimenti sono stati eseguiti da undici attori professionisti, sei uomini e cinque donne, scelti per coprire un indice di massa corporea (BMI) da 17 a 29, in modo da fornire una moderata variabilità per quanto riguarda la forma del corpo e i diversi movimenti.

1.2.2 Moulding Humans

“Moulding Humans” [3] usa immagini 2D e, tramite una GAN genera delle mappe di profondità che sono usate per ricostruire la forma della persona. Stima le profondità dalla foto senza fare troppa distinzione tra clothed e non clothed, a differenza del HMR.

1.2.3 BodyNet

La rete neurale proposta, BodyNet [12], è una rete neurale in grado di produrre una stima della forma del corpo volumetrico, prendendo in input una singola immagine. Si tratta di una rete addestrabile end-to-end che beneficia di (i) una loss volumetrica 3D, (ii) una loss di riproiezione multi-view e (iii) supervisione intermedia della posa 2D, della segmentazione delle parti del corpo 2D e della posa 3D. Per la *evaluation* del metodo, viene adattato il modello SMPL all'output della rete BodyNet che prende in ingresso il dataset SURREAL² [13] ed il dataset di Unite the People [6]. Tale rete è composta, dunque, da quattro moduli: (i) un modulo per l'estrazione della posa 2D, (ii) un modulo per estrarre una possibile segmentazione 2D, dopodiché (iii) i due moduli precedenti vengono combinati insieme all'immagine originale, permettendo di ottenere la posa 3D, e, infine, (iv) usando i 4 ingressi descritti precedentemente si ottiene in output la forma volumetrica, che viene usata come riferimento per ottimizzare il modello SMPL (che ottiene la unclothed shape).

1.2.4 Parsing clothing in fashion photograph

Nonostante l'abbigliamento determini gran parte della shape di una persona, ci sono stati relativamente pochi tentativi di riconoscimento computazionale dei capi di abbigliamento: ci si è focalizzati principalmente sull'identificazione delle parti del corpo, divise in superiore ed inferiore, per poi rappresentare l'abbigliamento come una deformazione del contorno del corpo sottostante. Di recente sono stati fatti dei tentativi per considerare i capi di abbigliamento come attributi semantici di una persona, ma solo per quanto riguarda un numero limitato di indumenti (come t-shirt, jeans, pantaloncini, ecc). A differenza di tali approcci, in “Parsing clothing in fashion photographs” [14] si cercano di stimare in maniera precisa le aree della superficie relative ad un'ampia lista di possibili capi indossati da un soggetto (come scarpe, cappello, cintura, calzini, ecc).

Per tale analisi, viene utilizzato un dataset che comprende 158'235 foto, con relativa descrizione, stile ed occasione per la quale l'outfit è stato indossato, ed un modello di riconoscimento che permette la segmentazione dei singoli capi di abbigliamento. Per semplificare tale processo, si assume che ogni capo di abbigliamento corrisponde ad una particolare area della superficie del corpo. In questo studio, la posa è fondamentale, in quanto può causare occlusioni importanti e di conseguenza complicare il riconoscimento degli abiti, per tale ragione viene effettuata anche una stima della posa a partire dall'immagine.

²<https://www.di.ens.fr/willow/research/surreal/data/>



Figure 1: (a) Superpixels (b) Stima della posa (c) Segmentazione dei capi di abbigliamento (d) Stima della posa finale

1.2.5 Multi-Garment Net

In “Multi-Garment Net: Learning to Dress 3D People from Images” [2] viene introdotta la rete multi-indumento (MGM, Multy-Garment Network), ovvero il primo modello in grado di posizionare dei layers esterni di vestiti (con mesh esclusive) direttamente sopra all’immagine data in input. Come si può vedere nella figura sottostante, questa nuova rappresentazione permette il pieno controllo sulla forma del corpo, sulla struttura e geometria dell’abbigliamento e apre le porte ad una vasta gamma di applicazioni VR/AR, intrattenimento, cinematografiche e virtual try-on.



Figure 2: Da sinistra a destra: immagini dal soggetto sorgente, corpo dal soggetto target, target vestito con indumenti originali. Da uno o più immagini, MGN può ricostruire la forma del corpo e ciascuna di esse i capi separatamente. Possiamo trasferire i capi previsti a un corpo nuovo che include geometria e texture.

Per raggiungere questo livello di controllo, MGM affronta due sfide principali:

1. apprendimento di un modello per ogni abito da scansioni tridimensionali di persone vestite
2. imparare a ricostruire tali modelli dalle immagini stesse

Per fare ciò, viene definito un insieme discreto di templates di abbigliamento (in base alle varie categorie: maglie lunghe/corte, pantaloni lunghi/corti e cappotti) e viene registrato, per ogni categoria, un singolo template per ciascuna delle istanze delle scansioni che vengono automaticamente segmentate in parti di vestito e pelle. Poiché la geometria dell'indumento varia in modo significativo all'interno di una categoria (ad es. forme diverse, lunghezze delle maniche), per prima cosa viene ridotto al minimo la distanza tra il modello ed i bordi della scansione, cercando di preservare il laplaciano della superficie del modello. In questo modo viene compilato un guardaroba digitale di veri capi 3D indossabili dalle persone (come mostrato sotto in Figura).



Figure 3: Viene utilizzato l'approccio di registrazione multi-mesh proposto per registrare i capi presenti nelle scansioni (a sinistra) su modelli di indumenti fissi. Questo permette di costruire un guardaroba digitale e vestire soggetti arbitrari (al centro) selezionando i capi (segnati) dall'armadio.

A partire da tali registrazioni, viene introdotto un modello PCA basato sui vertici degli abiti. Siccome gli indumenti sono associati (naturalmente) al sottostante modello SMPL [7], è possibile usarli su diverse morfologie del corpo per poi riposizionarli usando SMPL.

Dal guardaroba digitale, MGM è addestrato a fare previsioni partendo dai seguenti possibili input:

- una o più immagini della persona,
- posa del corpo,
- parametri della forma corporea.

I coefficienti PCA di ciascuno dei capi oltre che ad un “campo di variazione” sopra al modello PCA che codifica i dettagli dell'abbigliamento. Al momento del test, vengono perfezionate queste stime bottom-up con un nuovo obiettivo top-down che costringe gli indumenti e la pelle proiettati ad analizzare la segmentazione semantica dell'input.

Per apprendere un modello in grado di prevedere la forma del corpo e la geometria degli indumenti direttamente dalle immagini è necessario processare un dataset di 356 scansioni di persone con un'ampia varietà di vestiti, pose e morfologie.

Per quanto riguarda la pre-processazione dei dati abbiamo:

- registrazione SMPL alle scansioni,
- scansione e segmentazione del corpo,

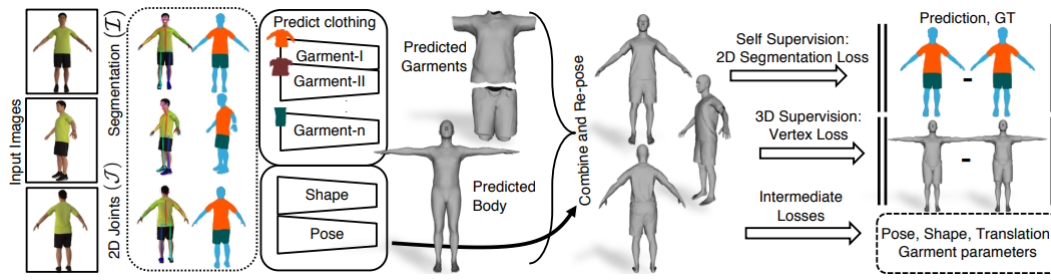


Figure 4: Dato un piccolo numero di frame RGB (attualmente 8), vengono pre-calcolate immagini segmentate semanticamente (I) e Giunti $2D(J)$. Il Multi-Garment Network (MGN), prende I, J come input e deduce gli indumenti separabili e il sottostante forma umana in una posa canonica. Queste previsioni vengono riposte usando le previsioni di posa per fotogramma. Viene fornito la MGN con una combinazione di Supervisione 2D e 3D. La supervisione 2D può essere utilizzata per il perfezionamento online al momento del test.

- registrazione del template.

Per ogni scansione viene ottenuta la forma del corpo al di sotto degli abiti e gli indumenti della persona, registrati in uno dei 5 modelli d'abbigliamento:

- camicia,
- t-shirt,
- cappotto,
- pantaloni corti,
- pantaloni lunghi.

I modelli di abbigliamento sono definiti come regioni sulla superficie SMPL dove la forma originale segue quella del corpo umano ma si deforma adattandosi ad ogni istante di scansione avvenuta dopo la registrazione.

In questo modo viene formata (sotto training process) la MGN per stimare la forma del corpo e gli indumenti da una o più immagini di una persona.



Figure 5: Viene utilizzato MGN per l'estrazione dei vestiti da un soggetto sorgente (immagine centrale) e successivamente questi abiti vengono messi su persone arbitrarie in diverse pose grazie a SMPL. I set corrispondono a quello maschile (sinistra) e quello femminile (destra)

Gli studi sui vestiti sono tanti, per fare in modo che un essere umano possa indossare virtualmente un abito che si adatti realisticamente ai propri movimenti in base al tipo di tessuto che

si vuole rappresentare, bisogna fare un lavoro di processing dal punto di vista della tassellazione non banale. In queste esperienze si arriva ad avere una mesh poligonale irregolare, di conseguenza una volta ottenuta la segmentazione non è possibile utilizzare separatamente gli abiti rispetto al resto del modello.

1.3 Approcci 3D

I lavori che partono da scansioni 3D o sequenze di scansioni 3D che sono stati presi in considerazione sono: BUFF, SizerNet e CAPE, ma di seguito vengono illustrati anche due ulteriori metodi rilevanti.

1.3.1 SPSD

In “A Layered Model of Human Body and Garment Deformation” [8], viene presentato un framework per l’apprendimento di un modello a tre livelli: forma del corpo umano, posa e deformazione degli indumenti. Il modello di deformazione proposto (Shape and Pose Space Deformation, SPSD) fornisce un controllo intuitivo ed indipendente sui tre parametri, producendo deformazioni naturalistiche sia degli abiti sia del corpo umano. I layer di deformazione della forma e della posa del modello sono addestrati su un ampio dataset di 520 scansioni 3D di 115 soggetti umani (57 uomini e 57 donne) in varie pose [4]. Il layer della deformazione degli indumenti, invece, è addestrato su sequenze di mesh animate di attori vestiti e si basa su una tecnica per la stima della forma e della postura umana sotto i vestiti. Il contributo chiave di tale articolo è la considerazione delle deformazioni degli abiti come trasformazioni residue tra una mesh di un soggetto svestito ed una della stessa persona vestita. L’inizializzazione della posa richiede un input manuale.

1.3.2 BUFF

In “Detailed, accurate, human shape estimation from clothed 3D scan sequences” [15] si parte da scansioni statiche 3D dimensionali, o sequenze di scansioni 3D, e si cercano di stimare posa e forma al di sotto dei vestiti. Se tali scansioni contengono informazioni sul colore, queste vengono usate per dividere i vertici delle scansioni in pelle e indumento, altrimenti tutti i vertici vengono considerati appartenenti ai vestiti. Viene usato il metodo di segmentazione descritto in ClothCap [9]. Per rendere uniforme la funzione di costo, viene calcolata prima la distanza geodetica tra un punto e quello del tessuto più vicino, per poi applicare una funzione logistica per mappare i valori della distanza geodetica compresi tra 0 e 1 (Fig. 3 b). Il valore risultante viene propagato ai punti di scansione mediante nearest distance e utilizzato per pesare ogni residuo di scansione. In questo modo, i punti vicini al confine pelle-tessuto hanno un peso decrescente uniforme. Ciò rende effettivamente la funzione non discontinua (*smooth*) e robusta a segmentazioni imprecise.



Figure 6: Valore dei pesi dati alla pelle.

- a) segmentazione ed allineamento (rosso: pelle, blu: vestiti)
- b) distanza geodetica dal vertice del tessuto più vicino relativo all’allineamento
- c) risultato ‘sbagliato’ con collo e braccio non generalizzato
- d) risultato finale

Il dataset BUFF³ contiene 11'054 clothed scans ad alta risoluzione con la corrispondente ground truth naked shape per ogni soggetto.

1.3.3 DeePSD

Con DeePSD [1] viene proposto un metodo semplice che consente di animare ogni template relativo ad un outfit indipendentemente dalla topologia, dagli strati da cui è composto e dalla complessità geometrica. Inoltre tale metodologia consente di generalizzare anche per quanto riguarda indumenti per i quali la rete neurale non è stata addestrata. Ciò che permette la generalizzazione è la codifica dei vestiti in un sottoinsieme di vertici del corpo sottostante. Questo è possibile identificando il resizing e l'animation come task separati.



Figure 7: Anteprima del dataset di DeePSD

L'architettura fa un mapping tra gli outfit ed i modelli standard 3D animati (blend weights e matrici di blend shapes). Il metodo proposto funziona con outfit completi (invece dei singoli indumenti considerati solitamente), più strati di vestiti e differenti risoluzioni, permettendo inoltre di avere dettagli geometrici complessi. Tutto ciò è possibile mediante l'utilizzo di una rete neurale di dimensioni ridotte, che permette quindi di ottenere:

- Generalizzazione degli outfit, difatti si tratta dell'unico lavoro in grado di animare outfit mai visti senza training ulteriore, il che rende possibili eventuali applicazioni in scenari in cui vi è un numero crescente di accostamenti di vestiti, come i virtual try-on ed i videogiochi, dove la chiave è la personalizzazione.
- Compatibilità, il metodo anziché predire la posizione dei vertici dei vestiti, ne predice i blend weights e le blend shapes matrices, rendendolo compatibile con tutti i motori grafici.
- Consistenza, cioè a differenza dei lavori simili che necessitano di una fase di post-processing per risolvere le collisioni, qui viene effettuato l'addestramento di un modello indipendente in modo tale che la physical consistency loss e la supervised loss non si ostacolino tra loro. Questo porta una quasi assenza di collisioni.
- Explainability, il mapping dei vestiti con i modelli di animazione 3D forniscono una pipeline più intuitiva per gli artisti CGI, a differenza dei lavori recenti che cercano di editare e fare il resizing degli indumenti attraverso una codifica con una rappresentazione parametrica, il che è possibile solo se si ha un'ottima conoscenza di tali parametri.

Predizione di modelli animati 3D

Due sono i metodi utilizzati nell'animazione di modelli 3D in computer graphics, e sono l'utilizzo dello *skinning* e/o delle *blend shapes*.

³<https://3dmd.com/tag/buff-dataset/>

1. **Skinning** Data una mesh 3D con N vertici $T \in \mathbb{R}^{N \times 3}$ ed uno scheletro con K giunti $J \in \mathbb{R}^{K \times 3}$, ogni vertice della mesh è attaccato ad ogni giunto con una matrice di pesi blend $W \in \mathbb{R}^{N \times K}$. L'animazione della mesh 3D può essere ottenuta trasformando lo scheletro J attraverso matrici di trasformazione lineare (rotazione, scala e traslazione). Le matrici di trasformazione dei vertici sono ottenute come una media pesata delle joint transformation. Per un'animazione realistica di uomo e vestiti viene applicata solo la rotazione.
2. **Blend Shapes** Presa T definita come sopra, una matrice di blend shapes $D \in \mathbb{R}^{M \times N \times 3}$ codifica M trasformazioni differenti (shapes) $D_i \in \mathbb{R}^{N \times 3}$ di T . Per animare la mesh, sono necessarie M shape keys. Tali chiavi sono usate per combinare linearmente le blend shapes allo scopo di ottenere la deformazione finale per T . L'evoluzione temporale delle shape keys anima la mesh.

Modelli 3D più complessi utilizzano una combinazione di entrambe le tecniche. Prima T viene deformato linearmente con le blend shapes e successivamente adattato allo scheletro J secondo i blend weights. Quando le shape keys sono definite in funzione della posa dello scheletro, si parla di Deformazione dello Spazio della Posa guidata dalle pose keys:

$$V_\theta = W \left(T + \sum_i^M f(\theta)_i D_i, J, \theta, W \right),$$

dove $W(\cdot)$ è la funzione di skinning che posa i vertici delle mesh come descritto da J e θ , V_θ sono i posed vertices, $f(\cdot)$ è la funzione che mappa la posa θ con le M pose keys e D_i sono le shape della matrice di blend shapes. Un esempio di questo approccio è SMPL.

Metodo

Viene proposto un addestramento di una rete neurale M per poter ottenere un mapping partendo da una mesh con un outfit template in posa canonica per ottenere i corrispondenti blend weights e blend shape matrices:

$$M : \{T, F\} \rightarrow \{W, D_{PSD}\},$$

dove T sono i vertici degli outfit e F le facce delle mesh, W e D_{PSD} sono i blend weights e le blend shapes matrices definite sopra.

Un template outfit viene processato dalla rete solo una volta per ottenere il formato standard del modello 3D animato. Una volta ottenuti blend weights e blend shapes matrices, l'outfit è usato come ogni altro modello 3D animato. Questo rende le predizioni automaticamente compatibili con tutti i motori grafici, inoltre si tratta di una rappresentazione estremamente efficiente computazionalmente. Il che estende ulteriormente la sua applicabilità a dispositivi postatili ed ambienti con minor efficienza computazionale. Presi PBS (Physically Based Simulation) data per gli indumenti indossati dai soggetti (SMPL) in sequenze di azioni differenti, vengono definiti i campioni $S = X, Y$ con $X = T, F, \theta, \beta, g$ e $Y = V_{PBS}$, dove T sono i vertici del template dell'outfit (posa canonica), F le facce della mesh, θ la posa dello scheletro, β i parametri della shape del corpo, g il genere e V_{PBS} la posizione dei vertici dell'outfit nei simulated data. L'obiettivo è quello di trainare M come definito nell'equazione precedente, in modo tale che W e D_{PSD} restituiscano V_{PBS} dopo aver applicato l'equazione dei posed vertices (da notare che per SMPL, lo scheletro è una funzione della shape β e del genere).

Il mapping tra lo spazio delle pose e quello degli outfit è una funzione multivalore. Diversi simulatori, condizioni iniziali, velocità di azione, timesteps ed integratori, tra gli altri fattori, genereranno diverse posizioni dei vertici degli outfit valide per una stessa shape, posa ed uno stesso outfit. La physical consistency è una parte cruciale dell'animazione di indumenti, ma altri approcci addestrano su dati PBS (che significa assumere che il mapping sia single-valued), per quanto ciò

sia utile nell'addestramento delle reti, minimizzare l'errore Euclideo rispetto al ground truth non garantisce la consistenza fisica, inoltre le applicazioni reali delle predizioni sono limitate. Pertanto viene proposta una combinazione tra addestramento supervisionato e addestramento physically based non supervisionato, per ridurre la necessità del post-processing. Inoltre rappresentare i vestiti come sottoinsiemi dei vertici del corpo fa perdere il template originale dell'outfit dopo averlo registrato in relazione al corpo, ma il modello proposto consente l'animazione indipendentemente dal resizing, senza perdere il template originale.

Architettura

L'architettura scelta deve essere in grado di:

- Gestire mesh non strutturate (senza vertex order e connettività fissati)
- Computare deformazioni non lineari rispetto alla posa θ (in quanto il comportamento dei vestiti è altamente non lineare).

Per fare questo, bisogna definire i seguenti componenti: $\Phi : \mathbb{R}^{N \times 3} \rightarrow \mathbb{R}^{N \times F}$, $\Omega : \mathbb{R}^{N \times F} \rightarrow \mathbb{R}^{N \times K}$, $\Psi : \mathbb{R}^{N \times F} \rightarrow \mathbb{R}^{P \times N \times 3}$, $\chi : \mathbb{R}^{N \times F} \rightarrow \mathbb{R}^{P \times N \times 3}$.

Φ computa descrittori F -dimensionali ad alto livello per ogni vertice con informazioni locali e globali della mesh template dell'outfit (con $F = 512$), Ω computa i blend weights per ogni vertice partendo dai descrittori dei vertici, Ψ genera una matrice di blend shapes mediante addestramento supervisionato (equivalente alle matrici di blend shapes per-vertice $D \in \mathbb{R}^{P \times 3}$) e χ genera una matrice di blend shapes mediante approccio non supervisionato che permette di avere consistenza fisica. Inoltre si noti che vengono definite P pose keys per le matrici di blend shapes al posto della dimensionalità della posa θ .

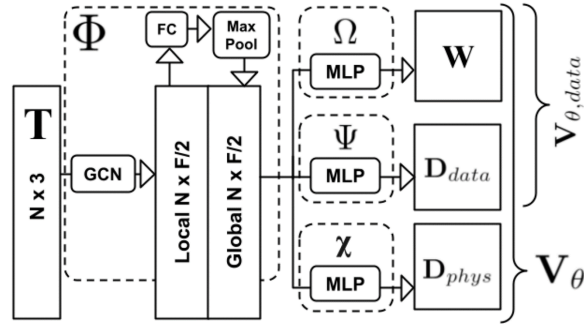


Figure 8: Architettura di DeePSD

θ viene passato attraverso un MLP per ottenere un embedding della posa ad alto livello $\Theta \in \mathbb{R}^P$. Questo per:

- controllare la dimensionalità P e, successivamente, la dimensione e la capacità della matrice di blend shapes;
- permettere la modellazione di non linearità dallo spazio della posa allo spazio dei vertici.

Per addestrare Φ vengono usati 4 livelli di graph convolutions applicati alla mesh template. Questo restituirà un descrittore locale, senza informazioni globali. Dopodiché, prendendo spunto da PointNet, vengono processati i descrittori attraverso un livello aggiuntivo fully-connected e tutti i descrittori dei vertici di ogni outfit vengono aggregati con il max pooling. Il descrittore globale viene concatenato ad ogni descrittore locale, dopodiché Ω , Ψ e χ vengono definite come MLP, ognuno con 4 livelli fully-connected, applicati ai descrittori dei vertici (i vertici sono campioni indipendenti). Poi Ω e χ computano entrambi le matrici di blend shape: D_{data} per minimizzare la

supervision loss e D_{phys} per la physical consistency. Nonostante siano indipendenti, le due matrici vengono combinate per ottenere la matrice PSD finale $D_{PSD} = D_{data} + D_{phys}$, mantenendo la compatibilità con i graphics engines detta prima. Infine, l'MPL usato per ottenere Θ consiste in 4 livelli fully-connected. L'output del modello durante l'addestramento è $V(\Theta, data)$ per D_{data} e V_{θ} per D_{PSD} . L'architettura scelta permette di fare il processing di mesh non strutturate con qualsiasi numero di vertici, ordine e connettività. Questo rappresenta un vantaggio significativo rispetto agli approcci che si basano sul modello del corpo per la rappresentazione dei vestiti.

1.3.4 Sizer

In "SIZER: A Dataset and Model for Parsing 3D Clothing and Learning Size Sensitive 3D Clothing" [11] ci si focalizza sulla vestibilità di vestiti virtuali su scansioni 3D del corpo. Gli indumenti interagiscono con il corpo in modo complesso, e la vestibilità è una funzione non lineare di taglia e shape del corpo, per cui simulare tale vestibilità non è banale. Viene introdotto il dataset SIZER⁴, un dataset di circa 2000 scansioni 3D di 100 persone che indossano 10 tipi di vestiario in taglie diverse (S, M, L, XL), registrazioni sul modello SMPL, scansioni segmentate in tipi di vestiario, categorie di vestiti e taglie. Con SIZER viene addestrata una rete neurale, SizerNet, che permette di stimare e visualizzare la vestibilità dei vestiti in base alla taglia, la human body shape e l'indumento dati in input. L'addestramento di SizerNet necessita di un mapping tra le scansioni e mesh multi-strato - mesh separate per il corpo e per i capi di sopra e di sotto. Per fare ciò, c'è bisogno di segmentare le scansioni 3D, stimare la body shape senza vestiti e registrare gli indumenti attraverso il dataset ottenuto tramite una feature extraction effettuata mediante i metodi spiegati in Multi-garment net[2] e ClothCap [9]. Dalle mesh multi-livello viene addestrato un codificatore in grado di mappare la mesh data in input ad un codice, ed un decodificatore che prende i parametri della forma del corpo di SMPL, la taglia dei vestiti dati in input e la taglia desiderata, per predire la vestibilità dei vestiti della taglia desiderata. Le scansioni, tuttavia, sono solo nuvole di punti, e analizzandoli (parsing) in una rappresentazione multi-layer al momento del test usando il metodo spiegato precedentemente (nel todo), richiede segmentazione, la quale può avere bisogno di un intervento manuale. Per questo è stata proposta anche ParserNet, la quale mappa automaticamente la registrazione di una singola mesh ad una mesh multi-layer con un solo passo feed-forward. ParserNet non solo segmenta la singola mesh registration, ma riparametrizza la superficie in modo da renderla coerente con i template dei vestiti più comuni. La rappresentazione multi-layer di ParserNet permette dunque di modificare i vestiti direttamente da una mesh data in input, eliminando la necessità di una segmentazione delle scansioni. Vengono quindi utilizzate: una rete neurale che predice i parametri relativi alla posa, una per predire i parametri della forma, ParserNet, che fa la segmentazione (riconosce il corpo e i singoli vestiti) e SizerNet, che modifica la taglia dei vestiti: prende in input una mesh 3D e restituisce in output corpi 3D svestiti ad alta qualità, mantenendo l'identità del soggetto.

1.3.5 SMPL

SMPL (Skinned Multi-Person Linear Model) [7] si pone come obiettivo quello di rappresentare corpi umani animati realistici che possano compiere diverse pose, deformandosi in maniera completamente naturale mantenendo però la fisionomia e la struttura del corpo umano che tutti noi conosciamo.

Tali modelli devono essere precisi, facili da implementare, veloci da renderizzare e compatibili con i motori di editor/rendering preesistenti sul mercato. Per realizzare tutto sono necessarie molte blend shapes, molto problematiche da realizzare perché richiedono un enorme sforzo manuale e altrettanto investimento temporale. Per ovviare al problema, la comunità di ricerca scientifica si è concentrata sull'apprendimento di modelli statici come per esempio numerose scansioni dello

⁴<http://virtualhumans.mpi-inf.mpg.de/sizer/>

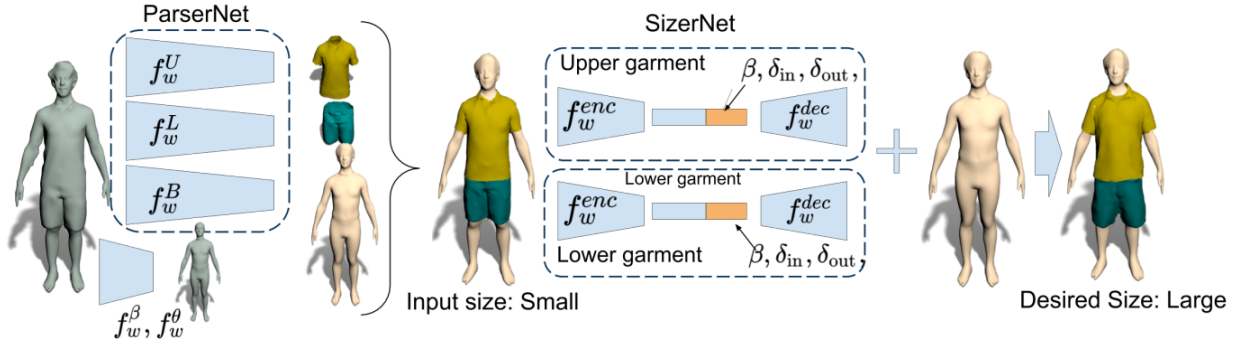


Figure 9: Viene proposto un modello per stimare e visualizzare l’effetto che viene a formarsi vestendo capi condizionati dalla forma e dalla taglia del corpo. Per fare questo viene introdotto ParserNet (f_w^U, f_w^L, f_w^B), che prende una mesh precedentemente registrata da SMPL $M(\theta, \beta, \mathbf{D})$ come input e prevede i parametri SMPL (θ, β) analizzando gli indumenti 3D ed utilizzando dei modelli predefiniti $T^g(\beta, \theta, \mathbf{0})$ prevedendo la forma del corpo che si cela sotto ai vestiti continuando a preservare i dati personali del soggetto. Viene inoltre proposto SizerNet, una rete encoder-decoder ($f_w^{\text{enc}}, f_w^{\text{dec}}$) che ridimensiona il capo ($\delta_{\text{in}}, \delta_{\text{out}}$) e lo adatta alla forma del corpo.



Figure 10: SMPL è un modello realistico appreso della forma e della posa del corpo umano che è compatibile con i motori di rendering esistenti, consente controllo dell’animatore ed è disponibile per scopi di ricerca. (a sinistra) Modello SMPL (arancione) adatto a mesh 3D della verità a terra (grigio). (a destra) Unità 5.0 screenshot del motore di gioco che mostra i corpi del dataset CAESAR animati in tempo reale.

stesso corpo in pose diverse. Inizialmente si è rivelata un’alternativa molto promettente ma, con l’avanzare delle tecnologie, questo metodo si è rivelato non compatibile con i vari software grafici presenti sul mercato e con i vari motori di rendering che utilizzano i metodi di skinning classici. SMPL descrive, appunto, un modello del corpo umano che possa rappresentare una vasta gamma di pose, con variazioni dipendenti dalla posa stessa mostrando il funzionamento delle dinamiche dei tessuti molli mantenendo l’efficienza e la compatibilità con i motori di rendering esistenti. I metodi tradizionali modellano il modo in cui i vertici sono correlati ad una struttura scheletrica sottostante. Il modello più utilizzato è senza ombra di dubbio il “Basic Linear Blend Skinning” (LBS) il quale è supportato da tutti i game engines, nonché efficiente da renderizzare; sfortunatamente però produce deformazioni delle pose completamente irrealistiche rispetto alle articolazioni umane, producendo i famosi effetti “taffy” e “papillon”

Un lavoro straordinario è stato dedicato sia ai metodi di skinning che cercano di migliorare questi effetti (Lewis et al. 2000; Wang and Phillips 2002; Kavan and Z̃ara 2005; Merry et al. 2006; Kavan et al. 2008), sia per migliorare l’apprendimento di modelli corporei altamente realistici dai dati (Allen et al. 2006; Anguelov et al. 2005; Freifeld and Black 2012; Hasler et al. 2010; Chang and Zwicker 2009; Chen et al. 2013). Nonostante gli enormi sforzi nell’implementazione di metodi, sono rimasti annidati molti problemi tra cui:

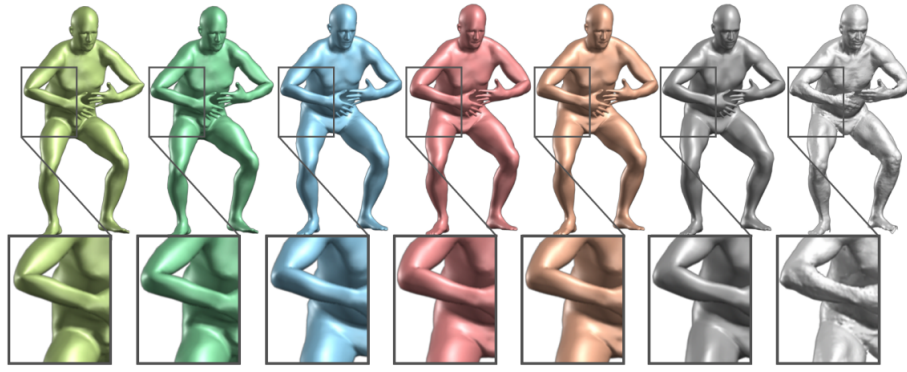


Figure 11: Modelli a confronto con verità di base. Questa figura definisce la codifica a colori utilizzata nel documento e nel video supplementare. La mesh all'estrema destra (grigio chiaro) è una scansione 3D. Accanto ad essa (grigio scuro) c'è una mesh registrata con la stessa topologia del nostro modello. Chiediamo come ben diversi modelli possono approssimare questa registrazione. Da sinistra a destra: (verde chiaro) Skinning blend lineare (LBS), (verde scuro) Skinning blend Dualquaternion (DQBS), (blu) BlendSCAPE, (rosso) SMPL-LBS, (arancione) SMPL-DQBS. Le regioni ingrandite evidenziano le differenze tra i modelli al gomito destro e all'anca del soggetto. LBS e DQBS producono gravi artefatti su ginocchia, gomiti, spalle e fianchi. BlendSCAPE ed entrambi i modelli SMPL si comportano allo stesso modo bene nell'adattare i dati.

- mancanza di realismo
- non funzionamento con i pacchetti esistenti
- non rappresentazione di una discreta varietà di pose
- non compatibilità con le pipeline grafiche standard
- richiesto un notevole lavoro manuale

Contrariamente agli approcci precedenti, un obiettivo chiave del progetto SMPL è quello di rendere il modello del corpo il più semplice e standard possibile in modo che possa essere ampiamente utilizzato, mantenendo allo stesso tempo il realismo di modelli basati sulla deformazione appresi dai dati. Nello specifico vengono implementate diverse blend shapes per correggere i limiti dello skinning tradizionale. Blend shapes per identità, posa e dinamiche dei tessuti molli vengono combinate in modo additivo con un modello di riposo prima di essere trasformate da una blend skin. Per imparare a riconoscere i vari portamenti del corpo, SMPL utilizza 1786 scansioni in alta risoluzione con un'ampia varietà di pose. Viene inoltre utilizzata la PCA (principal component analysis) per imparare i modelli lineari maschili e femminili dal dataset CAESAR (con approssimativamente 2000 scansioni maschili e femminili).

Inizialmente viene registrata una mesh template per ogni scansione e ne viene quindi normalizzata la posa (processo molto importante quando si sta imparando da un modello vertex-based shape).

Successivamente viene allenato il modello SMPL in diversi modi e confrontato quantitativamente con un simile modello BlendSCAPE [Hirshberg et al. 2012], allenato anch'esso con gli stessi dati. I 2 modelli vengono poi valutati qualitativamente con animazioni e quantitativamente utilizzando delle mesh mai viste durante la fase di training. SMPL e BlendSCAPE vengono adattati a queste nuove mesh e viene fatta un'analisi sugli errori di vertice. Sono state esplorate principalmente 2 varianti del modello SMPL:

1. la prima utilizzando LBS (linear blend skinning)

2. la seconda usando DQBS (Dual-Quaternion blend skinning)

Risulta che un modello skinned vertex-based come SMPL è molto più accurato rispetto a un modello deformation-based come il BlendSCAPE (avendo svolto la fase di training sugli stessi dati). Il modello SMPL è stato anche esteso per catturare e modellare la dinamica dei tessuti “molli” adattando il Dyna model [Pons-Moll et al. 2015]. Il risultante modello “Dynamic-SMPL” o DMPL, viene addestrato dallo stesso set di dati di mesh 4D di Dyna. DMPL, a differenza di Dyna, si basa su vertici anziché su deformazioni triangolari. Successivamente vengono calcolati ed analizzati gli errori di vertice tra le mesh SMPL e Dyna, e tramite l'utilizzo di PCA viene ridotta la dimensionalità producendo delle blend shapes dinamiche. Un modello di tessuto “molle” viene addestrato sulla base di velocità angolari e deformazioni dinamiche con fatto in [Pons-Moll et al. 2015]. In questo genere di tessuti, la dinamica dipende fortemente dalla forma del corpo; DMPL viene addestrata usando corpi con variabili indici di massa corporea e concentrandosi nel training che dipenda fortemente dai movimenti e pose del corpo stesso.

Linear Blend Skinning

La skin blend lineare è l'idea di trasformare i vertici all'interno di una singola mesh mediante una (blend) di più trasformazioni. Ogni trasformazione è la concatenazione di una “matrice di legame” che porta il vertice nello spazio locale di un dato “osso” e una matrice di trasformazione che si sposta dallo spazio locale di quell'osso in una nuova posizione. Verrà usata una shader di questo tipo:

Blend Shapes

Le Blend Shapes permettono di trasformare la forma di un oggetto in un altro, più nello specifico consentono di trasformarne la superficie stessa. Per esempio, un uso ricorrente delle Blend Shapes riguarda l'animazione facciale:

1.3.6 CAPE

CAPE (https://cape.is.tuebingen.mpg.de/media/upload/CAPE_paper.pdf) è un modello generativo strutturato su grafi a base di reti neurali convoluzionali per la realizzazione di mesh tridimensionali specifiche per l'adattamento dei vestiti al corpo umano. È compatibile con i più famosi modelli che analizzano il corpo umano, come per esempio SMPL, e può essere generalizzato a particolari pose e movimenti del corpo. È progettato per essere “plug-and-play” per molte applicazioni che già utilizzano SMPL. Il dataset di CAPE fornisce delle mesh registration nel metodo SMPL quadridimensionali di scansioni relative a persone vestite, insieme a scansioni veritiere delle forme del corpo spoglie di ogni vestito.

Per modellare i corpi vestiti, CAPE li fattorizza in due parti:

1. corpo con pochi vestiti
2. layer vestito rappresentato come un offset dal corpo

Classificando in questo modo, CAPE è in grado di estendere in maniera completamente naturale SMPL ad una classe di tipi di abbigliamento trattando il vestito come una shape aggiuntiva. Grazie alla rapida diffusione di SMPL, l'obiettivo di CAPE è diventato quello di estendere i metodi implementati da SMPL in maniera coerente agli attuali usi.

Come già precedentemente spiegato, SMPL è un modello che fattorizza la superficie del corpo nei parametri forma (β) e posa (θ). Come mostrato nella figura sottostante (sezione a,b), l'architettura di SMPL inizia con il template di una mesh triangolare $\bar{T}$, definita da $N = 6890$ vertici. Dopo aver dato al modello i parametri forma e posa (β, θ), vengono aggiunti al template gli offset 3D corrispondenti a deformazioni dipendenti


```

uniform mat4 mvp;
uniform mat4x3 bones[40]; //make sure this is more than the number
of bones
in vec4 Position;
in vec3 Normal;
in vec4 Color;
in vec2 TexCoord;
in vec4 BoneWeights;
in uvec4 BoneIndices;
out vec3 normal;
out vec4 color;
out vec2 texCoord;
void main() {
    vec3 skinned =
        BoneWeights.x * ( bones[ BoneIndices.x ] * Position)
        + BoneWeights.y * ( bones[ BoneIndices.y ] * Position)
        + BoneWeights.z * ( bones[ BoneIndices.z ] * Position)
        + BoneWeights.w * ( bones[ BoneIndices.w ] * Position);
    gl_Position = mvp * vec4(skinned, 1.0);
    vec3 skinned_normal = inverse(transpose(
        BoneWeights.x * mat3(bones[ BoneIndices.x ])
        + BoneWeights.y * mat3(bones[ BoneIndices.y ])
        + BoneWeights.z * mat3(bones[ BoneIndices.z ])
        + BoneWeights.w * mat3(bones[ BoneIndices.w ]) )) *
Normal;
    normal = skinned_normal;
    color = Color;
    texCoord = TexCoord;
}

```

- dalla forma ($B_S(\beta)$)
- dalla posa ($B_P(\theta)$)

La posa della mesh risultante viene ottenuta usando una funzione di skinning W . Formalmente abbiamo:

$$\begin{aligned}
 T(\beta, \theta) &= \bar{T} + B_S(\beta) + B_P(\theta) \\
 M(\beta, \theta) &= W(T(\beta, \theta), J(\beta), \theta, \mathcal{W})
 \end{aligned} \tag{1}$$

dove la funzione di sfumatura della pelle $W(-)$ ruota i vertici T della posa a “riposo” intorno ai giunti tridimensionali J (calcolati grazie a β), li “leviga” linearmente grazie ai pesi di fusione W e restituisce i vertici della nuova posa M .

SMPL aggiunge dei livelli (strati) di deformazione lineare ad uno strato iniziale composto dalla posa “standard” del corpo. Successivamente, CAPE definisce l’abbigliamento come un livello extra di offset dal corpo e lo aggiunge sopra alla mesh realizzata da SMPL, come in Fig. 2 (d).

Architettura della rete

Come mostrato nell’immagine sottostante, il modello è costituito da un generatore G con architettura encoder-decoder e discriminatore D . Vengono inoltre utilizzate reti ausiliarie $C1$, $C2$ per gestire il problema del condizionamento. La rete è differenziabile ed è trainata end to end.

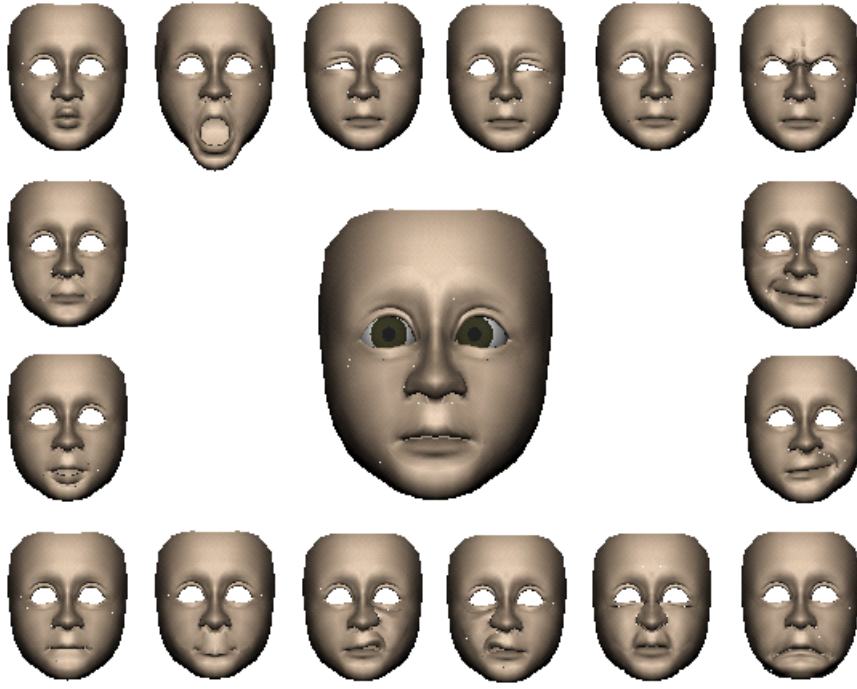


Figure 12: Quando viene creata una blend shape, si identificano uno o più oggetti che si vogliono deformare. Gli oggetti utilizzati per “prendere” la forma interessata sono chiamati oggetti target, mentre l’oggetto deformato è chiamato oggetto base.

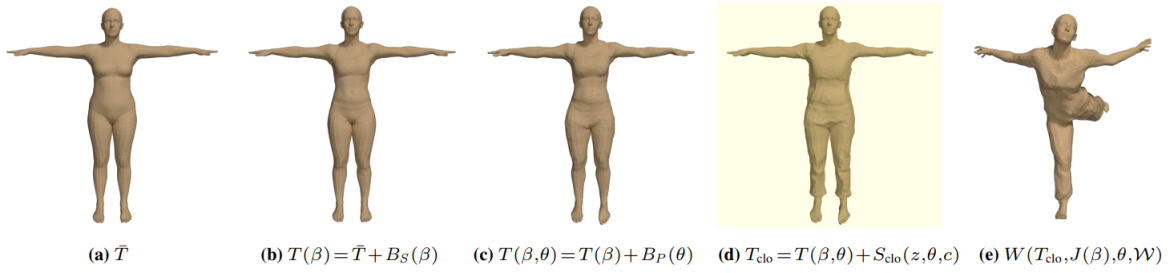


Figure 13: Il contributo principale viene evidenziato dallo sfondo giallo. Partendo da SMPL, CAPE (a) aggiunge linearmente gli offset forniti da (b), la forma del corpo individuale ($/\beta$) e (c) la posa ($/\theta$); da notare la deformazione sui fianchi e sui piedi per via della posa. (d) Viene inoltre aggiunto uno strato di abbigliamento c e una variabile di forma z . (e) i vertici vengono computati usando l’equazione di skinning di SMPL.

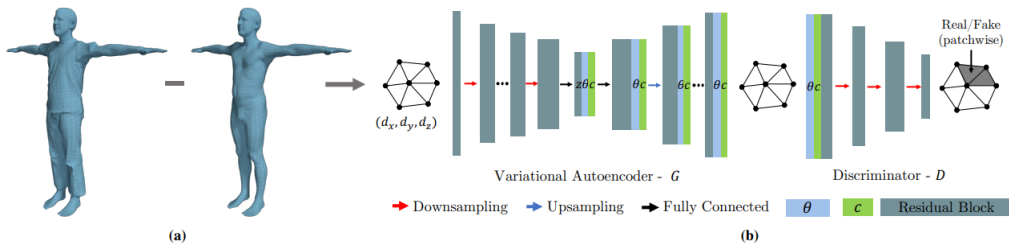


Figure 14: (a) calcola gli spostamenti dai dati di scansione sottraendo la minima forma del corpo dalla mesh del corpo vestito (b)

Per semplicità, in questa sezione verrà utilizzata la seguente notazione.

x : i vertici ν_d del grafo di spostamento in ingresso;

X : vertici del grafo ricostruito;
 θ e c : la posa ed il vettore di condizionamento in base al capo
 z : il codice latente.

Generatore di grafici. Viene costruito il generatore di grafi seguendo una rete di tipo VAE-GAN. Durante l'allenamento, un encoder $\text{Enc}(\cdot)$ prende lo spostamento x , estrae le sue caratteristiche attraverso più strati convoluzionali del grafico e lo mappa sul codice latente a bassa dimensione z . Un decoder viene addestrato per ricostruire il grafico di input $\hat{x} = \text{Dec}(z)$ da z . Sia l'encoder che il decoder sono reti neurali feed-forward costruite con strati convoluzionali mesh. I livelli lineari sono usati alla fine dell'encoder e all'inizio del decoder.

L'impilare i livelli convoluzionali provenienti dal grafico genera una perdita di features negli strati più profondi. Questo è un gravissimo problema, soprattutto nell'ambito del clothing generation perchè alcuni dettagli, come per esempio le pieghe, tendono a scomparire. Pertanto, vengono migliorati gli strati di convoluzione del grafico standard con connessioni residue, che consentono l'uso di caratteristiche di basso livello da quello di input, se necessario.

Al momento del test, l'encoder non è necessario. Invece, z viene campionato dalla distribuzione a priori gaussiana e dal decodificatore funge da generatore di grafici: $G(z) = \text{Dec}(z)$.

Discriminatore Patchwise. Per migliorare ulteriormente i dettagli nella fase di ricostruzione, viene introdotto un discriminatore patchwise D per i grafici, che ha mostrato successo nel dominio dell'immagine.

Invece di guardare l'intero grafico generato, il discriminatore classifica solo se la patch è vera o falsa in base alla sua struttura locale. Intuitivamente questo incoraggia il discriminatore a concentrarsi solo sui dettagli più raffinati e per quanto riguarda la forma 'globale', se ne occupa la reconstruction loss.

Viene implementato il grafico patchwise-discriminator utilizzando quattro grafici di convoluzione-sottocampionamento a blocchi. Successivamente viene aggiunto una perdita reale/fasulla discriminante per ciascuno dei vertici di output. Questo abilita il discriminatore a catturare una patch di nodi vicini nel grafico ricostruito e classificarli come reali / falsi.

Modello condizionale. Si condiziona la rete con bodypose θ e abbigliamento di tipo c . I parametri di posa SMPL sono nella rappresentazione asse-angolo e sono difficili da apprendere per la rete neurale. Pertanto vengono trasformati i parametri di posa in matrici rotazionali usando l'equazione di Rodrigues. Entrambe le condizioni v vengono prima passate attraverso una piccola rete di incorporamento completamente connessa, $C_1(\theta)$, $C_2(c)$, rispettivamente, in modo da bilanciare la dimensionalità di apprendimento delle features del grafico e di condizionamento.

Si sperimenta, inoltre, diversi modi di condizionare il generatore di mesh:

Concatenazione nello spazio latente; aggiungere le features delle condizioni alle features del grafico in tutti i nodi del generatore; e la combinazione dei due. Risulta che la strategia combinata funziona meglio in termini di capacità di rete e di effetto del condizionamento.

Ricostruzione di persone 3D

La ricostruzione di esseri umani 3D da immagini e video 2D è un classico problema di visione artificiale. La maggior parte degli approcci generano mesh del corpo 3D dalle immagini, ma non dai vestiti. Questo tipo di approccio ignora completamente dettagli delle immagini che potrebbero essere utili ai fini dell'obiettivo stesso. Per ricostruire i corpi con i vestiti addosso, i più famosi metodi utilizzano rappresentazioni di profondità volumetriche o biplanari per modellare il corpo e gli abiti nel loro insieme. Un altro gruppo di metodi si basa sul SMPL. Rappresentano l'abbigliamento come uno strato sfalsato rispetto al corpo sottostante come proposto in ClothCap (gruppo 2 nella Tabella 1).

Modelli parametrici per corpi e abiti 3D

I modelli statistici tridimensionali del corpo umano ricavati dalle scansioni 3D catturano la forma e la posa, rendendoli un elemento fondamentale per molteplici applicazioni. Nella maggior parte dei casi, tuttavia, le persone sono vestite quando questi modelli non rappresentano l'abbigliamento stesso. Inoltre, mentre ci muoviamo, i vestiti cambiano forma producendo pieghe che a loro volta possono mutare in molti modi diversi. Sebbene esistano modelli di abbigliamento appresi da dati reali, pochi di questi si adattano a pose non comuni. Ad esempio, Neophytou e Hilton apprendono un modello basato sulla stratificazione degli indumenti sulla base di SCAPE da sequenze dinamiche, anche se la generalizzazione a nuove pose non è ancora stata dimostrata. Un approccio concettualmente diverso deduce i parametri di un modello di abbigliamento da sequenze di scansioni tridimensionali. In questo caso è possibile generalizzare a nuove pose creando, però, il grave problema dell'interferenza e, a differenza di CAPE, il simulatore fisico risultante non è differenziabile rispetto ai parametri stessi.

Il dataset CAPE è facilmente scaricabile, sono approssimativamente 49GB. Per ottenere i dataset di Buff e SIZER bisogna necessariamente contattare gli owners. Per Buff bisogna inviare una email seguendo le istruzioni del seguente link: <http://buff.is.tue.mpg.de/downloads>. Per quanto riguarda SIZER, invece, c'è un form⁵ da compilare, oppure si può inviare una mail a gtiwari@mpi-inf.mpg.de per ottenere sia il dataset che il modello⁶.

La cattura, ricostruzione e la modellazione di abiti sono 3 punti che sono stati ampiamente studiati negli ultimi decenni. La tabella 1 mostra i più famosi metodi degli ultimi anni categorizzati in due classi principali: (1) metodi di ricostruzione e cattura e (2) modelli parametrici, dettagliati come segue.

Method Class	Methods	Parametric Model	Pose-dep. Clothing	Full-body Clothing	Clothing Wrinkles	Captured Data*	Code Public	Probabilistic Sampling
Image Reconstruction	Group 1 [†]	No	No	Yes	Yes	Yes	Yes	No
	Group 2 [‡]	Yes	No	Yes	Yes	Yes	Yes	No
Capture	ClothCap [40]	Yes	No	Yes	Yes	Yes	No	No
Clothing Models	DeepWrinkles [28]	Yes	Yes	No	Yes	Yes	No	No
	Yang et al. [52]	Yes	Yes	Yes	No	Yes	No	No
	Wang et al. [51]	Yes	No	No	No	No	Yes	Yes
	DRAPE [16]	Yes	Yes	Yes	Yes	No	No	No
	Sanesteban et al. [45]	Yes	Yes	Yes	Yes	No	No	No
	GarNet [18]	Yes	Yes	Yes	Yes	No	No	No
	Ours	Yes	Yes	Yes	Yes	Yes	Yes	Yes

Figure 15: Selezione di metodi correlati. Esistono due principali classi di metodi di abbigliamento 3D: (1) metodi di ricostruzione e acquisizione basati su immagini e (2) abbigliamento modelli che prevedono la deformazione in funzione della posa. All'interno di ciascuna classe, i metodi differiscono in base ai criteri nelle colonne.

1.3.7 ClothCap

In “ClothCap: Seamless 4D Clothing Capture and Retargeting” [9] viene utilizzato un approccio data-driven alla cattura dei vestiti, processando abbigliamento dinamico su persone da scansioni quadridimensionali, in modo da poter essere computato in maniera più semplice dei tradizionali metodi e applicato su avatar virtuali.

Viene catturata la geometria dell'abito su un corpo in movimento, viene stimata sia la forma del corpo che la sua morfologia sotto i vestiti per poi segmentare ed estrarre i vari capi. I nuovi

⁵https://docs.google.com/forms/d/e/1FAIpQLSddBep3Eif1gI-6IhaZybBDoR-_H_QW1NST0JV5vviauVpNTA/viewform

⁶<https://github.com/garvita-tiwari/sizer>

capi catturati in questo modo vengono assegnati successivamente a nuove persone, anche con pose differenti.

Per fare ciò, viene sviluppato un modello multi-parte orientato alle mesh dimostrando come segmentare, rintracciare e recuperare la forma di un capo a partire da sequenze di scansioni tridimensionali.



Figure 16: Da sinistra a destra: (1) Un esempio di scansione con texture 3D che fa parte di una sequenza 4D. (2) Il modello di mesh allineato in più parti, sovrapposto al corpo. (3) La forma minimamente vestita (MCS) stimata sotto i vestiti. (4) Il corpo irrobustito e vestito con gli stessi abiti. Si noti che l'abbigliamento si adatta in modo naturale alla nuova forma del corpo. (5) Questa nuova forma del corpo si pone in una posa nuova, mai vista. Questo illustra come ClothCap supporta una gamma di applicazioni relative all'acquisizione, alla modellazione, al retargeting, alla posa e alla prova di abbigliamento

I metodi di cattura esistenti soffrono di bassa risoluzioni, forme statiche, troppa semplicità nei movimenti del corpo, cattura di solo un capo di abbigliamento o la non completa segmentazione dei vestiti. Conseguentemente, i problemi chiave da risolvere includono l'acquisizione di alta qualità, la segmentazione, il tracciamento della forma e della superficie così come la stima della morfologia del corpo e come questo stia posando.

Come input, viene data una nuvola di punti (con texture).

La rappresentazione multi-mesh è consistente con il modo in cui l'abito viene indossato, modellando e adattando il continuo cambiamento di visibilità delle superfici del vestito nel tempo. Inoltre permette di affrontare i problemi di segmentazione e tracciamento in un quadro coerente. In questo modo vengono segmentati automaticamente gli abiti dalle scansioni quadridimensionali usando informazioni sia riguardanti l'aspetto che la forma (Fig. 1 (2)).

Dati gli abiti segmentati, viene adattato un set di modelli multiparte basati a mesh alle scansioni, mettendoli in corrispondenza nel tempo e stimando così la forma sottostante del vestito in forma minimale.

La filosofia di questo metodo sposta l'attenzione dalla simulazione alla cattura dei vestiti. Le mesh dei vestiti estratte automaticamente sono naturalmente associate ad un modello corporeo articolato sottostante (SMPL (Loper et al. 2015)). Grazie a questa tecnologia si è in grado di cambiare la morfologia del corpo come mostrato in (Fig. 1 (4)), oppure trasferire direttamente gli indumenti ad un nuovo corpo in maniera completamente automatica. È vero che l'acquisizione del modello riguardante le pieghe non è completamente realistico; tuttavia l'aspetto visivo ottenuto è più che sufficiente per molte applicazioni try-on.

In sintesi, ClothCap getta le basi per la modellazione degli abiti, affrontando problemi complessi relativi alla segmentazione e tracciamento di scansioni quadridimensionali. In maniera più dettagliata, contribuisce a

- metodo automatico per segmentare le sequenze di scansioni tridimensionali sfruttando un modello corporeo

- un metodo di tracciamento del modello multi-mesh
- una tecnica per riorientare il tessuto in maniera dinamica ad una nuova morfologia corporea

L'intero metodo viene dimostrato estraendo diverse tipologie di abiti tra cui pantaloni lunghi, t-shirt e pantaloncini su un'ampia varietà di persone mentre eseguono movimenti non banali (complessi). Viene inoltre dimostrato che il metodo funziona per le gonne pur avendo una diversa topologia rispetto al corpo.

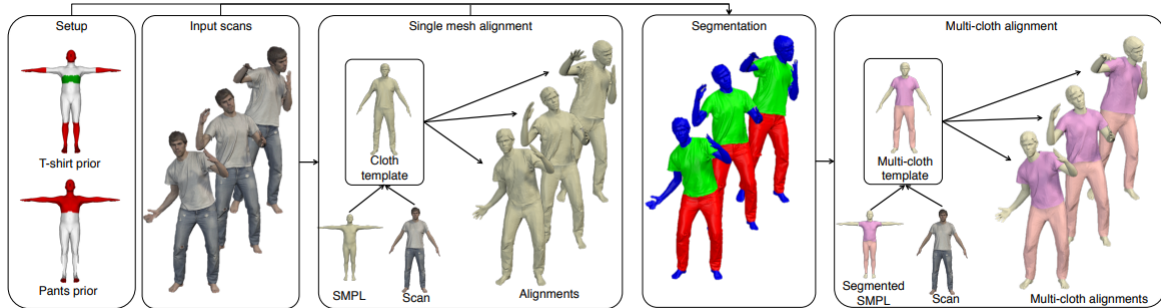


Figure 17: Sono necessari tre passaggi principali per ottenere allineamenti multitessuto: allineamento mesh singola, segmentazione della scansione e allineamento multitessuto. Multi-tessuto gli indumenti di allineamento possono essere facilmente retargettati a nuove forme.

ClothCap si basa su quattro steps principali:

- **Step 0: Installazione.** Per catturare una specifica classe di abiti è necessario definire il numero di capi “Ngarm” e istanziare una priorità spaziale relativamente bassa per intuire in quale parte del corpo gli abiti potrebbero trovarsi. Questo processo è molto grezzo e conseguentemente facile da realizzare. Inizialmente vengono trattati vestiti la cui topologia è facilmente mappabile al corpo per poi venire esteso ad un più ampio campo di vestiti con topologie via via più complicate come per esempio le gonne. Come modello del corpo viene utilizzato l’SMPL. (Loper et al. 2015). Nello specifico, per un soggetto, viene stimata una MCS (minimally clothed shape) ovvero la morfologia approssimativa del soggetto senza vestiti. L’MCS potrebbe essere stimato utilizzando una tecnologia esistente (Zhang et al. 2017) ma, dal momento che ClothCap acquisisce scansioni quadridimensionali dal soggetto, l’acquisizione di un’ulteriore scansione per inquadrare gli abiti “di base” richiederebbe minimo sforzo a livello computazionale e tempistico. Quindi viene allineato il modello del corpo insieme alla scansione dei vestiti basilari per ottenere l’ MCS. Infine ClothCap assume che tutte le sequenze di scansione partano (più o meno) con una posa alla “A”.
- **Step 1: Segmentazione.** Le scansioni quadridimensionali includono geometria e informazione riguardante il colore e, ovviamente, contengono anche rumore e mancanza di dati. Risulta, quindi, difficile segmentare queste scansioni grezze pertanto il problema viene analizzato in due sottoproblemi:
 - **Step 1a: Allineamento a mesh singola.** Come primo passo viene allineato alle sequenze scansionate il modello del corpo umano fornito da SMPL. Successivamente ClothCap deforma il modello in modo da adattarlo al meglio ai dati osservati e inizializzare ciascun frame con il risultato del frame precedente. L’output di questo step è una sequenza registrata a bassa risoluzione di meshes.

- **Step 1b: Segmentazione.** Attraverso l'utilizzo combinato della priority segmentation e un campo casuale Markoviano è possibile segmentare ogni frame della sequenza. Ciò fornisce una segmentazione sia dei singoli allineamenti della mesh sia una segmentazione basata sui vertici delle scansioni. Quindi grazie all'allineamento segmentato della singola mesh del primo frame (che, come citato sopra, si trova approssimativamente in una posa ad "A") è possibile definire un template che determini la topologia del capo (ma non la geometria). Infine vengono scartati gli allineamenti della singola mesh.
- **Step 2: Allineamento multitessuto.** Il template segmentato dell'abito viene deformato in modo da adattarsi alla scansione segmentata del primo fotogramma in modo da ottenere un modello multitessuto che è in grado di acquisire sia la topologia che la geometria dei capi e del corpo umano. Con questo tipo di allineamento ogni vestito viene tracciato per combaciare con i dati della scansione segmentata. Questa ottimizzazione implementa un parametro per regolarizzare i bordi del capo come per esempio nelle braccia, sul collo, nella vita e nelle caviglie. Inoltre un ulteriore parametro viene dedicato allo smoothing degli abiti. Questo step genera multi tessuti allineati alle scansioni, ovvero deformazioni del template multitessuto originale. Ne conclude che ogni abito in ogni fotogramma è allineato ad un template comune.
- **Step 3: Reindirizzamento.** Per prima cosa bisogna considerare la sorgente di una persona che si sta muovendo con abiti che vogliamo reindirizzare. Da questa sequenza viene stimato l'MCS e computato un allineamento multitessuto; questo, però, non è sufficiente per l'operazione di reindirizzamento. Nella fase di preparazione viene stimata la morfologia di una persona svestita in un determinato lasso di tempo che potrebbe discostarsi dal MCS e viene stimato come il vestiario si possa adattare a questa nuova forma del corpo nella posa a "T". Questa operazione viene svolta per ogni singolo fotogramma nella sequenza sorgente. Il reindirizzamento di un capo prevede 2 steps:
 - preparazione
 - vestizione

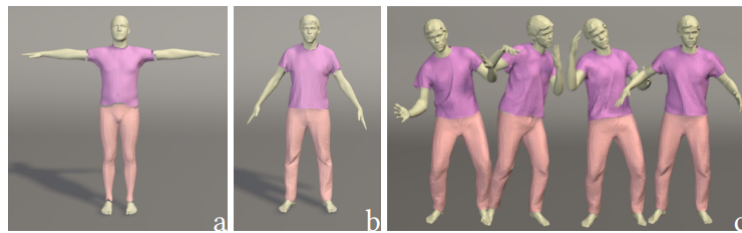


Figure 18: modello di tessuto segmentato automaticamente: definisce il solo topologia dell'indumento; cioè, quali vertici appartengono a ciascuna parte di stoffa. (b) il modello multi-tessuto acquisisce sia la geometria che la topologia. Questo è calcolato per ogni nuovo soggetto e capo e cattura la geometria di ciascuno indumento. (c) gli allineamenti multi-tessuto sono deformazioni del multi-telo modello per adattarsi a ogni fotogramma della sequenza.

2 Obiettivi

Dalla conoscenza ottenuta durante la fase di ricerca della letteratura, sono emersi i seguenti obiettivi:

- ottenere le mesh dei singoli vestiti partendo da scansioni 3D
- animare delle scansioni
- animare i vestiti in real time

Tali obiettivi possono essere accorpati nei seguenti macroargomenti:

- Ricostruzione 3D.
Oltre a migliorare la pipeline 3D, è fondamentale il processo di *delighting*, cioè ricavare il colore di un oggetto indipendentemente dalle condizioni di luce ambientale.
- Modelli deformabili.
Si intende lavorare sui corpi umani e vedere come i modelli si deformano, compresa la componente del vestito. L'obiettivo è dunque di separare la componente del corpo e quella del vestito vedendo l'indumento come un offset dai vertici del corpo umano. Partendo da una immagine comprendente un soggetto vestito, l'idea è quella di avere il modello 3D del corpo umano nudo e quello delle componenti del vestito. Un problema che si può riscontrare, però, è la limitatezza della tipologia di vestiti presi in considerazione solitamente.

Questo capitolo sulla modellazione dei vestiti è ciò che ci interessa maggiormente: si può ricavare la modellazione del vestito a partire dal reale, cioè dall'osservazione di qualcuno che indossa tali indumenti, o, in alternativa, si può lavorare direttamente sulle immagini o su scansioni 3D di soggetti vestiti, in modo da non dover avere l'onere di gestire la parte di proiezione sull'immagine, andando a lavorare sulla parte di sola modellazione. Una volta ottenute le mesh separate, bisogna re-mesharle per applicare algoritmi di deformazione di vestiti.

I modelli tridimensionali del corpo umano sono ampiamente utilizzati nell'analisi della posa e del movimento umano. I modelli esistenti, tuttavia, vengono processati da scansioni 3D con abiti di base, non riuscendo quindi a generalizzare la complessità delle persone che indossano qualcosa di più dettagliato, non riuscendo a studiare il modello nella sua interezza. Inoltre, gli attuali modelli mancano della forza espressiva necessaria per rappresentare la complessa geometria non lineare di indumenti dipendenti dalla posa delle forme. Modellare ed analizzare come l'abbigliamento 3D si adatta e interagisca con il corpo umano in funzione di diverse strutture porta ad avere numerose applicazioni nel settore tecnologico odierno, in quanto parte di una consistente fetta applicativa relativa ai contenuti 3D (come per esempio film, videogiochi, settore AR/VR etc.). Al giorno d'oggi è molto "rischioso" acquistare un capo online, in quanto non è possibile provarlo e vedere come viene calzato. Proprio per questo motivo la vendita di vestiti sul web è uno tra i pochi settori che non è ancora riuscito a contrastare la controparte fisica: le persone hanno bisogno di provare capi di persona. Si stima che i rivenditori di abiti online subiscano perdite per oltre 600 miliardi di dollari ogni anno a causa di vendite tornate indietro: è difficile acquistare abiti online senza poterli provare.

3 SIZER: dataset e modello per l'analisi e l'apprendimento dell'abbigliamento 3D size sensitive

Esistono numerosi modelli nell'ambito dell'abbigliamento 3Dimensionale, progettati ed ottimizzati grazie a database costruiti su basi di dati reali. Nessuno di questi, però, è in grado di implementare la funzione di "deformare" l'abito in base alla taglia.

- SizerNet in grado di prevedere le deformazioni 3D elaborate in tempo reale degli abiti condizionate in base ai parametri del corpo umano ed alle dimensioni dell'indumento
- ParserNet con l'obiettivo di decifrare e dedurre la struttura dei vestiti (e la loro forma), distinguendoli dal corpo umano tramite un singolo passaggio data una mesh in input.

SizerNet apre la possibilità di stimare e visualizzare nel dettaglio la vestibilità di un indumento nelle sue varie taglie mentre ParserNet permette di modificare vestiti relativi alla mesh di input in maniera diretta e precisa, eliminando la necessità di scannerizzare le singole segmentazioni, problema molto impegnativo già in sé.

In “SIZER: A Dataset and Model for Parsing 3D Clothing and Learning Size Sensitive 3D Clothing” [11] ci si focalizza sulla vestibilità di vestiti virtuali su scansioni 3D del corpo. Gli indumenti interagiscono con il corpo in modo complesso, e la vestibilità è una funzione non lineare di taglia e shape del corpo, per cui simulare tale vestibilità non è banale. Viene introdotto il dataset SIZER (<http://virtualhumans.mpi-inf.mpg.de/sizer/>), un dataset di circa 2000 scansioni 3D di 100 persone che indossano 10 tipi di vestiario in taglie diverse (S, M, L, XL), registrazioni sul modello SMPL, scansioni segmentate in tipi di vestiario, categorie di vestiti e taglie. Con SIZER viene addestrata una rete neurale, SizerNet, che permette di stimare e visualizzare la vestibilità dei vestiti in base alla taglia, la human body shape e l'indumento dati in input. L'addestramento di SizerNet necessita di un mapping tra le scansioni e mesh multi-strato - mesh separate per il corpo e per i capi di sopra e di sotto. Per fare ciò, c'è bisogno di segmentare le scansioni 3D, stimare la body shape senza vestiti. Dalle mesh multi-livello viene addestrato un codificatore in grado di mappare la mesh data in input ad un codice, ed un decodificatore che prende i parametri della forma del corpo di SMPL, la taglia dei vestiti dati in input e la taglia desiderata, per predire la vestibilità dei vestiti della taglia desiderata. A questo punto entra in gioco ParserNet, che mappa automaticamente la registrazione di una singola mesh ad una mesh multi-layer con un solo passo feed-forward. ParserNet non solo segmenta la singola mesh registration, ma riparametrizza la superficie in modo da renderla coerente con i template dei vestiti più comuni. La rappresentazione multi-layer di ParserNet permette dunque di modificare i vestiti direttamente da una mesh data in input, eliminando la necessità di una segmentazione delle scansioni. Vengono quindi utilizzate: una rete neurale che predice i parametri relativi alla posa, una per predire i parametri della forma, ParserNet, che fa la segmentazione (riconosce il corpo e i singoli vestiti) e SizerNet, che modifica la taglia dei vestiti: prende in input una mesh 3D e restituisce in output corpi 3D svestiti ad alta qualità, mantenendo l'identità del soggetto. Il codice, il modello e il set di dati sono stati rilasciati all'indirizzo <https://virtualhumans.mpi-inf.mpg.de/sizer/>.

4 Dataset

Il dataset SIZER⁷, specifico nella variazione di taglia degli indumenti, comprende 2000 scansioni di 100 soggetti con diversa forma fisica, che indossano ciascuno due o tre taglie (S, M, L o XL) di differenti stili di abbigliamento casual. Vi sono 10 classi di indumenti, vale a dire shirt, dress-shirt (che si differenziano per la lunghezza della camicia), jeans, hoodie, polo t-shirt, t-shirt, shorts, vest, skirt e coat, ognuna contenente circa 200 scansioni.

E' stato scelto di catturare le mesh dei soggetti in una A-pose in modo tale da avere una vestibilità il più naturale e senza pieghe possibile. Poiché le scansioni sono state acquisite mediante più di 130 fotocamere e ricostruite con il software *Agisoft's Metashape*, hanno alta risoluzione

⁷https://github.com/garvita-tiwari/sizer_dataset



Figure 19: (A sinistra): scansioni 3D di persone in diversi stili e taglie di abbigliamento. (Destra): maglietta e pantaloncini pantaloni per taglie piccole e grandi, che sono registrati su modelli comuni.

e i grafi delle diverse mesh che le rappresentano hanno differenti connessioni, quindi sarebbe difficoltoso utilizzare tale dataset, per questo motivo è stato necessario aggiungere al modello una fase di preprocessing del dataset, registrando le scansioni al modello SMPL.

SMPL rappresenta il corpo umano come una funzione parametrica di posa e forma. SMPL+G è una formulazione parametrica che permette di rappresentare corpo e vestiti come mesh separate. Per registrare gli indumenti bisogna per prima cosa segmentare le scansioni in modo da dividere le parti relative al corpo e quelle relative ai vestiti. Per ogni classe di vestiti si ottiene una mesh template definita come un subset del template di SMPL, dove ogni vertice dell'indumento è associato ad un vertice della forma fisica, come offset di quest'ultima.

Dunque, il dataset contiene scansioni, scansioni segmentate, registrazioni SMPL+G, classi di indumenti e le taglie come etichette, ed è stato costruito per costruire un modello per l'estrazione di vestiti da una singola mesh (ParserNet) ed un resizing dei vestiti stessi (SizerNet).

Per ottenere il dataset è necessario andare alla sezione Data⁸ del sito.

I soggetti delle scansioni di cui è composto il dataset hanno una età compresa tra i 19 e i 37 anni, con due sole eccezioni (51 e 69 anni). Per quel che riguarda l'altezza e il peso dei soggetti scansionati, invece, vi è un range piuttosto vasto, che permette una forte differenziazione dei dati.

Il dataset è composto da scansioni di sole A-pose. Tali scansioni sono suddivise per genere e per tipologia di indumento (che può essere a maniche lunghe, maniche corte, pantalone lungo e pantalone corto). Come si può vedere dalle immagini seguenti, sono presenti anche gli oggetti utili a capire quali sezioni del corpo umano escludere dall'estrazione della superficie relativa agli abiti.

Poiché il dataset SIZER include registrazioni sul modello SMPL, bisogna scaricare il modello SMPL⁹, che prende in input i parametri di forma e posa e restituisce il volume.

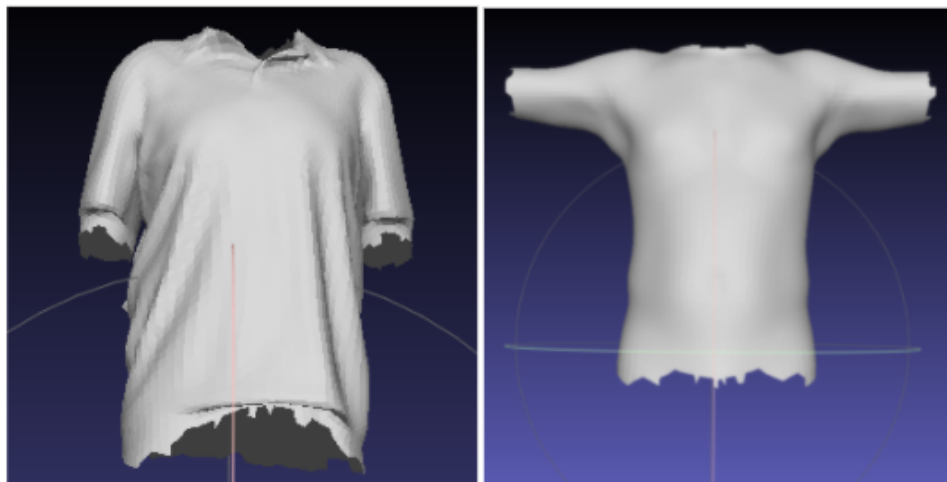
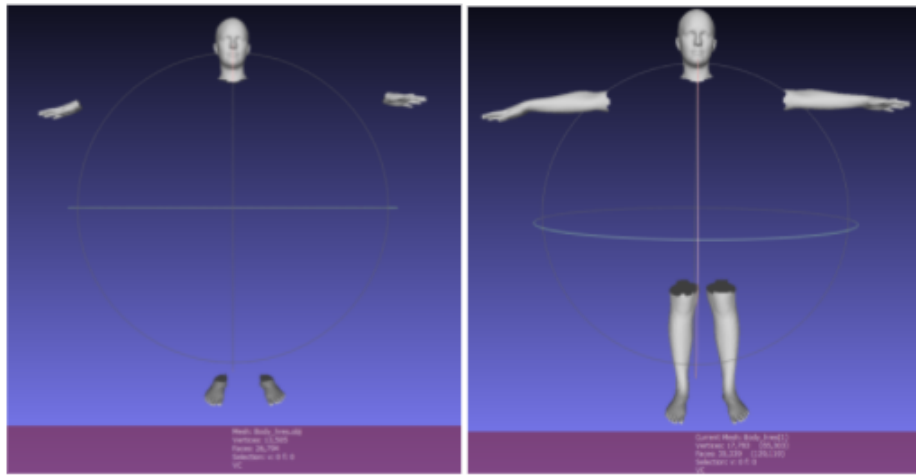
Preprocessing Il dataset SIZER è stato ottenuto tramite una feature extraction effettuata mediante due metodi illustrati di seguito nel dettaglio.

4.1 Confronto tra SIZER e altri datasets

Ad oggi, pochi dataset contengono scansioni di modelli tridimensionali relativi a soggetti e vestiti segmentati. Tra questi 3DPeople, Cloth3D consistono di un vasto dataset di rappresentazioni sintetiche di soggetti 3D vestiti. Ma non contengono deformazioni realistiche degli indumenti,

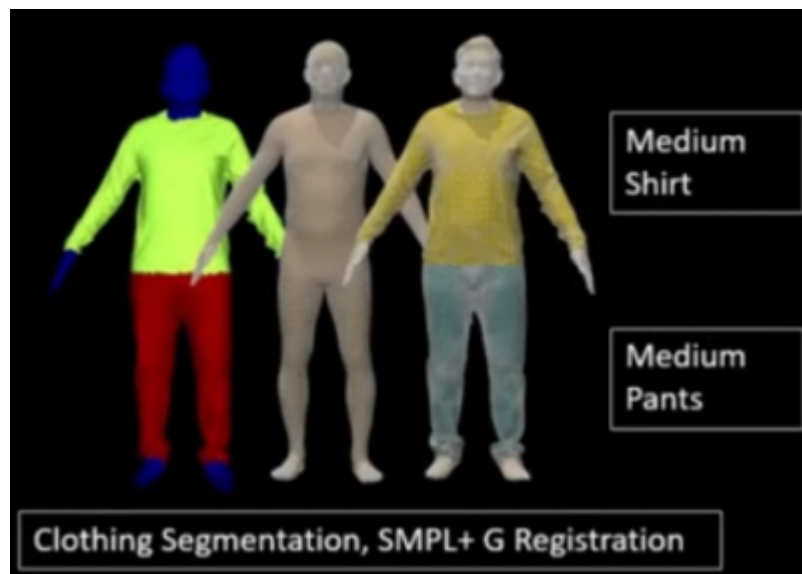
⁸<http://virtualhumans.mpi-inf.mpg.de/sizer/>

⁹<https://smpl.is.tue.mpg.de/>



realUpperClothes.obj

upperClothes.obj



come fa invece SIZER. THUman contiene sequenze di persone 3D con relativi abiti in movimento, catturati grazie ad un sensore RGBD (Kinectv2), e ricostruiti utilizzando un fusore volumetrico SDF. Tali scansioni, però, perdono di dettaglio rispetto alle scansioni 3D di SIZER e non vengono fornite le segmentazioni degli indumenti reali. Dyna e D-FAUST, invece, riguardano scansioni 3D ad alta risoluzione di 10 soggetti in movimento con forme differenti; l'unica pecca è che indossano solo vestiti "semplici". BUFF contiene scansioni ad alta risoluzione 3D di 6 soggetti con e senza abiti; neanche questo dataset tuttavia garantisce la segmentazione degli abiti: infatti è stato

ideato per addestrare modelli per stimare solo la body shape che si trova al di sotto dei vestiti. DeepFashion3D riguarda scansioni di vari tipi di vestiti e potrebbe essere sostituito a SIZER per l'addestramento della rete neurale.

5 Metodo

Siccome queste scansioni sono ad alta risoluzione e rappresentate da mesh con diverse connettività ai grafi sottolineanti (ovvero grafi dove ogni arco direzionato viene sostituito con un vertice privo di direzione) risulta complicato utilizzare tale dataset per qualsiasi modello di apprendimento. Per questo motivo è necessario eseguire un preprocessing registrando grazie all'utilizzo di SMPL.

Per migliorare il funzionamento generale del dataset SIZER, vengono implementate le registrazioni SMPL+G.

Le scansioni vengono registrate tramite SMPL ampiamente descritto nella prima parte della relazione, realizzando così una corrispondenza tra le scansioni del dataset e SMPL stesso, garantendo un maggior controllo sui parametri di posa e forma tramite sottolineatura fornita sempre da SMPL.

Vediamo ora un breve funzionamento di SMPL e SMPL+G specifico a SIZER:

SMPL rappresenta il corpo umano come funzione parametrica $M(\cdot)$, di posa (θ) e forma (β). Vengono aggiunti degli offset relativi ai vertici (D) in aggiunta ai modelli delle deformazioni SMPL in grado di gestire capelli, vestiti etc.

SMPL applica una forma base del corpo $W(\cdot)$ ad un template preimpostato T posto, appunto, in T-pose.

Da qui, W descrive una scala di pesi $B_p(\cdot)$ and $B_s(\cdot)$, deformazioni dei modelli relativi rispettivamente a posa e forma.

$$\begin{aligned} M(\beta, \theta, D) &= W(T(\beta, \theta, D), J(\beta), \theta, W) \\ T(\beta, \theta, D) &= T + B_s(\beta) + B_p(\theta) + D \end{aligned}$$

SMPL+G, invece, è una formulazione parametrica per rappresentare il corpo umano ed i vestiti in mesh separate.

Per registrare i vestiti vengono prima segmentate le scansioni relative alle varie parti del corpo.

Per ogni classe di indumenti viene ottenuto un template (sottoforma di mesh), definito come un subset del template generato da SMPL, dato da: $T^g(\beta, \theta, 0) = \mathbf{I}^g T(\beta, \theta, 0)$, dove $\mathbf{I}^g \in \mathbb{Z}_2^{m_g \times n}$ è un indicatore di matrice, con $\mathbf{I}_{i,j}^g = 1$ se i vertici i dell'abito in questione g $i \in \{1 \dots m_g\}$ sono associati con la corrispettiva forma del corpo tracciata dai vertici dati dalla scansione $j \in \{1 \dots n\}$. m_g e n denota il numero di vertici appartenenti al template del vestito e delle mesh SMPL rispettivamente. Similmente viene definita una funzione specifica all'abito $G(\beta, \theta, D^g)$ usando l'equazione descritta subito sotto, dove D^g sono gli offsets dei vertici ricavati dal template.

$$G(\beta, \theta, D^g) = W(T^g(\beta, \theta, D^g), J(\beta), \theta, W)$$

Il dataset preprocessato servirà a costruire il modello ParserNet per l'estrazione dei vestiti dalle singole mesh.

5.1 ParserNet

ParserNet è un metodo per l'estrazione dei vestiti partendo direttamente dalle mesh SMPL. Per fare ciò, è necessario predire i parametri SMPL del corpo sottostante utilizzando una rete per predire posa e forma del corpo umano. Successivamente si applica ParserNet per l'estrazione dei vari livelli di vestiti e di features quali capelli, caratteristiche facciali usate per modellare la body shape sotto i vestiti.

5.1.1 Rete per la predizione di posa e forma

Per stimare la body shape sotto i vestiti viene inizialmente creato un corpo svestito (tramite SMPL) per uno specifico input dato da una persona vestita fornito tramite mesh ad unico layer $M(\beta, \theta, \mathbf{D})$, prevedendo θ, β , usando rispettivamente f_w^θ and f_w^β . Viene applicato il processo di training f_w^θ e f_w^β con una perdita L_2 sui parametri e una perdita per-vertex tra la mesh di input e il corpo predetto da SMPL.

$$\begin{aligned}\mathcal{L}_\theta &= w_{\text{pose}} \|\hat{\theta} - \theta\|_2^2 + w_v \|M(\beta, \hat{\theta}, \mathbf{0}) - M(\beta, \theta, \mathbf{D})\| \\ \mathcal{L}_\beta &= w_{\text{shape}} \sum_{i=1}^{10} \sigma_i \left(\hat{\beta}_i - \beta_i \right)^2 + w_v \|M(\hat{\beta}, \theta, \mathbf{0}) - M(\beta, \theta, \mathbf{D})\|\end{aligned}$$

Siccome i parametri θ, β , relativi al corpo di riferimento possono essere imprecisi, viene aggiunta una perdita relativa ai vertici addizionale tra la mesh data input $M(\beta, \theta, \mathbf{D})$ ed i vertici del corpo predetto $M(\hat{\theta}, \hat{\beta}, \mathbf{0})$ in modo tale da ottenere un corpo svestito il più simile possibile a quello della mesh data in input.

Facendo il training di f_w^θ e f_w^β separatamente si osservano risultati più stabili, usando come riferimenti β e θ . Poichè le componenti di β in SMPL sono normalizzate (ovvero $\sigma = 1$), vengono denormalizzate scalando le loro corrispettive deviazioni standard $[\sigma_1, \sigma_2, \dots, \sigma_{10}]$ come dimostrato nell'equazione sottostante.

$$\begin{aligned}\mathcal{L}_\theta &= w_{\text{pose}} \|\hat{\theta} - \theta\|_2^2 + w_v \|M(\beta, \hat{\theta}, \mathbf{0}) - M(\beta, \theta, \mathbf{D})\| \\ \mathcal{L}_\beta &= w_{\text{shape}} \sum_{i=1}^{10} \sigma_i \left(\hat{\beta}_i - \beta_i \right)^2 + w_v \|M(\hat{\beta}, \theta, \mathbf{0}) - M(\beta, \theta, \mathbf{D})\|\end{aligned}$$

Qui, $w_{\text{pose}}, w_{\text{shape}}$ e w_v sono pesi relativi alla perdita su posa, forma e superficie SMPL predetta. $(\hat{\theta}, \hat{\beta})$ denotano i parametri predetti. Il risultato sarà una body shape svestita lineare.

Here, $w_{\text{pose}}, w_{\text{shape}}$ and w_v are weights for the loss on pose, shape and predicted SMPL surface. $(\hat{\theta}, \hat{\beta})$ denote predicted parameters. The output is a smooth (SMPL model) body shape under clothing.

5.2 Analisi dei vestiti

L'analisi dei vestiti ottenuti tramite una singola mesh (M) può essere ottenuta segmentando la mesh stessa in vestiti separati per ogni classe di appartenenza ($\mathbf{G}_{\text{seg}}^{g,k}$), che si traduce nella connettività di diversi grafo sottostante ($\mathcal{G}_{\text{seg}}^{g,k} = (\mathbf{G}_{\text{seg}}^{g,k}, \mathbf{E}_{\text{seg}}^{g,k})$) attraverso tutte le istanze (k) di un vestito appartenente alla classe g come si può vedere nella figura sottostante. Siccome tale procedimento rende difficile la generalizzazione del dataset in qualsiasi modello di apprendimento, pertanto viene proposto di effettuare il parsing dei vestiti deformando i vertici dei vestiti relativi al template T: $T^g(\beta, \theta, \mathbf{0})$ con connettività fissata $\mathbf{E}^g$, ottenendo i vertici $\mathbf{G}^{g,k} \in \mathcal{G}^{g,k}$, dove $\mathcal{G}^{g,k} = (\mathbf{G}^{g,k}, \mathbf{E}^g)$ sono visibili nella parte centrale della figura sottostante.

Si procede a prevedere i vertici deformati $\mathbf{G}^g$ come una combinazione convessa dei vertici presi dalla mesh di input iniziale $\mathbf{M} = M(\boldsymbol{\beta}, \boldsymbol{\theta}, \mathbf{D})$ utilizzando una matrice a regressione sparsa $\mathbf{W}^g$, in modo che risulti $\mathbf{G}^g = \mathbf{W}^g \mathbf{M}$. Specificatamente, ParserNet predice tale matrice sparsa ($\mathbf{W}^g$) come una funzione di features di mesh presa in input (vertici e normali) e un insieme di neighbor per ogni vertice i di una classe di vestiti g .

$$\mathbf{G}_i = \sum_{j \in \mathcal{N}_i} \mathbf{W}_{ij} \mathbf{M}_j$$

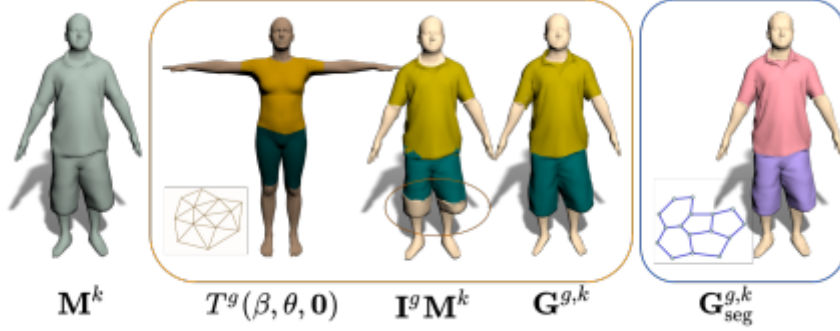


Figure 20: Da sinistra a destra: In input una singola mesh ($\mathbf{M}^k$), il template dell'abito ($T^g(\boldsymbol{\beta}, \boldsymbol{\theta}, \mathbf{0}) = \mathbf{I}^g T(\boldsymbol{\beta}, \boldsymbol{\theta}, \mathbf{0})$), estrazione della mesh del vestito usando $\mathbf{G}^{g,k} = \mathbf{I}^g \mathbf{M}^k$, mesh multilivello ($\mathbf{G}^{g,k}$) registrata a SMPL+G, tutto questo con classi di vestiti munite di specifiche connessioni tra vertici $\mathbf{E}^g$, e scansioni segmentate $\mathbf{G}_{\text{seg}}^{g,k}$ con specifiche istanze di connettività tra vertici $\mathbf{E}_{\text{seg}}^{g,k}$.

5.3 Analisi della body shape svestita

Ai fini di generare una dettagliata body shape svestita, viene prima creata una mesh del corpo lineare, grazie all'utilizzo dei parametri SMPL $\boldsymbol{\theta}$ e $\boldsymbol{\beta}$ ricavati da f_w^θ, f_w^β . Utilizzando la formulazione di combinazione convessa citata sopra, Body ParserNet trasferisce i vertici della pelle visibile dalla mesh data in input a quella lineare creata, ottenendo così le caratteristiche facciali e relative ai capelli. Successivamente viene scomposta la mesh di input in magliette, pantaloni e forma del corpo svestita utilizzando le 3 sottoreti di ParserNet (f_w^U, f_w^L, f_w^B) mostrate in figura 1.3.

5.4 Funzione di Loss

La rete ParserNet viene addestrata tramite la funzione di Loss sotto descritta:

$$\begin{aligned} \mathcal{L}_{\text{parser}} &= w_{3D} \mathcal{L}_{3D} + w_{\text{norm}} \mathcal{L}_{\text{norm}} + w_{\text{lap}} \mathcal{L}_{\text{lap}} + w_{\text{interp}} \mathcal{L}_{\text{interp}} + w_w \mathcal{L}_w \\ \mathcal{L}_{\text{sizer}} &= w_{3D} \mathcal{L}_{3D} + w_{\text{norm}} \mathcal{L}_{\text{norm}} + w_{\text{lap}} \mathcal{L}_{\text{lap}} + w_{\text{interp}} \mathcal{L}_{\text{interp}} \end{aligned}$$

dove $w_{3D}, w_{\text{norm}}, w_{\text{lap}}, w_{\text{interp}}$ and w_w sono i pesi relativi ai vertici di Loss, normali, Laplaciani, di interpenetrazione e di termini di regolarizzazione dei pesi.

5.4.1 3D Vertex Loss per vestiti

Viene definito $\mathcal{L}_{3D}$ come una Loss nello spazio L_1 tra vertici predetti ed effettivi.

$$\mathcal{L}_{3D} = \|\mathbf{G}_P - \mathbf{G}_{GT}\|_1.$$

5.4.2 3D Vertex Loss corpi svestiti

Per addestrare f_w^B (ParserNet specifica per il corpo), viene utilizzata la mesh di input relativa alla pelle come supervisione per prevedere i dettagli personali del soggetto. Si definisce un termine di loss pesato (tramite la distanza geodetica) specifico per ogni classe di vestiti.

$$\mathcal{L}_{3D}^{\text{body}} = \|\mathbf{w}_{\text{geo}}^T \cdot \text{abs}_{ij}(\mathbf{G}_P^s - \mathbf{I}^s \mathbf{M})\|_1$$

dove $\mathbf{I}^s$ è la matrice di indicatori per la specifica regione di pelle e $\mathbf{w}_{\text{geo}}$ è un vettore contenente il sigmoide del termine di loss specifico ottenuto tramite distanza geodetica dai vertici appartenenti alla regione relativa alla pelle e non. Il termine di Loss è elevato quando la predizione è molto diversa dalla mesh originale di input $\mathbf{M}$ relativa alle porzioni di pelle visibili, e bassa per quanto riguarda le regioni relative agli indumenti, con una transizione lineare regolata dal termine geodetico. Nella formulazione citata sopra, il termine $\text{abs}_{ij}(\cdot)$ rappresenta un operatore di valore assoluto.

5.4.3 Normal Loss

Viene definito $\mathcal{L}_{\text{norm}}$ come la differenza in angolo tra la faccia normalizzata reale ($\mathbf{N}_{GT}^i$) e quella predetta ($\mathbf{N}_P^i$).

5.4.4 Termine di smoothness Laplaciano

Si definisce $\mathbf{L}^g \in \mathbb{R}^{m_g \times m_g}$ come grafo Laplaciano relativo alla mesh dell'abito $\mathbf{G}_{GT}$, e $\Delta_{\text{init}} = \mathbf{L}^g \mathbf{G}_{GT} \in \mathbb{R}^{m_g \times 3}$ come le coordinate differenziali di $\mathbf{G}_{GT}$, successivamente si computa il termine di smoothness Laplaciano per una predetta mesh $\mathbf{G}_P$ come

$$\mathcal{L}_{\text{lap}} = \|\Delta_{\text{init}} - \mathbf{L}^g \mathbf{G}_P\|_2.$$

5.4.5 Interpenetration loss

Siccome minimizzare la perdita specifica dei vertici non garantisce che l'indumento predetto sia esterno alla superficie del corpo, viene utilizzato il termine di Interpenetration loss:

$$\mathcal{L}_{\text{interp}} = \sum_{(i,j) \in \mathcal{C}(\mathbf{B}, \mathbf{G}_P)} \mathbb{K}_{d(\mathbf{G}_{P,j}, \mathbf{G}_{GT,j}) < d_{\text{tol}}} \text{ReLU}(-\mathbf{N}_i(\mathbf{G}_{P,j} - \mathbf{B}_i)) / m_g,$$

Per ogni vertice $\mathbf{G}_{P,j}$ viene trovato quello più vicino all'interno della body shape svestita predetta $\mathbf{B}_i$ e viene definita una corrispondenza corpo-vestito $\mathcal{C}(\mathbf{B}, \mathbf{G}_P)$. Se il vertice relativo all'indumento $\mathbf{G}_{P,j}$ si trova all'interno della superficie del corpo, il modello viene penalizzato mediante la formula sopra descritta.

da notare che $\mathbb{K}_{d(\mathbf{G}_{P,j}, \mathbf{G}_{GT,j}) < d_{\text{tol}}}$ attiva il termine di loss solo quando la distanza tra i vertici della mesh del vestito predetta ed i vertici della mesh reale è piccola, ad esempio: $< d_{\text{tol}}$.

5.4.6 Regularizzazione dei pesi

Per mantenere i dettagli più particolari e fini mentre viene divisa la mesh in input si vuole che i pesi della rete siano sparsi e racchiusi all'interno di un unico cluster. Conseguentemente viene aggiunto un regolarizzatore che penalizza valori elevati per $\mathbf{W}_{ij}$ se la distanza tra $\mathbf{M}_j$ ed il vertice $\mathbf{M}_k$ (con peso maggiore $k = \arg \max_j \mathbf{W}_{ij}$) è grande. Viene definito $d(\cdot, \cdot)$ come distanza Euclidea tra i vertici, allora il regolarizzatore equivale a:

$$\mathcal{L}_w = \sum_{i=1}^{m_g} \sum_{j \in \mathcal{N}_i} \mathbf{W}_{ij} d(\mathbf{M}_k, \mathbf{M}_j), k = \arg \max_j \mathbf{W}_{ij}$$

5.5 Cenno sulle reti neurali utilizzate

Vengono implementate f_w^θ e f_w^β : reti con due livelli completamente connessi ed un livello di output lineare. Successivamente si implementa ParserNet f_w^U, f_w^L, f_w^B con 3 livelli completamente connessi. Nello specifico caso di SIZER sono stati utilizzati per gli esperimenti cluster ($\mathcal{N}_i$) di dimensione 50. Inizialmente viene addestrata la rete con classi di vestiti che condividono lo stesso template e successivamente si adattano separatamente per ogni classe g . Per velocizzare il processo di addestramento di ParserNet, si addestra la rete a prevedere $\mathbf{W}^g = \mathbf{I}^g$, dove $\mathbf{I}^g$ riguarda la matrice di indicatori per la classe di vestito g . Grazie a ciò, si inizializza il processo per il quale la rete suddivide il vestito tagliando parte della mesh di input basata sulla matrice di indicatori per ogni vestito.

Per la rete di predizione di posa e form, ParserNet usa una dimensione di batch = 8 e una curva di apprendimento di 0.0001.

6 Codice

Per utilizzare il codice (<https://github.com/garvita-tiwari/sizer>) sono necessari come prerequisiti le librerie mesh e kaolin. In caso di dubbi sui passaggi da seguire, di seguito vengono riportate le istruzioni in maniera semplificata.

Di seguito viene illustrata una guida alle installazioni a seconda del sistema operativo di cui si è in possesso.

7 Linux

I comandi mostrati successivamente sono stati testati su una distribuzione Ubuntu 20.04, con la versione di Python 3.6.8.

Per prima cosa, bisogna installare le librerie di Boost¹⁰, in quanto serviranno per l'utilizzo di Psbody-mesh.

- `$ sudo apt-get install libboost-dev`
- `$ sudo apt install python3.8-venv`
- `$ (opzionale) > mkdir python-venvs`

¹⁰https://www.boost.org/users/history/version_1_77_0.html

- *\$ (opzionale) > cd python-venvs*
- *\$ python3 -m venv --copies jvenv_name*
- *\$ source jvenv_name/bin/activate*
- *\$ mkdir workspace*
- *\$ cd workspace/*

La directory workspace è stata creata per far sì che tutte le librerie che si andranno ad installare si trovino in un unico ambiente di lavoro; durante la loro installazione, quindi, è importante trovarsi nella cartella principale del workspace.

7.0.1 Psbody-mesh

Questo pacchetto contiene le principali funzioni di manipolazione e visualizzazione di mesh. Richiede Python3.5+ e le librerie di Boost.

- *\$ sudo apt install git*
- *\$ git clone https://github.com/MPI-IS/mesh*
- *\$ cd mesh*

A questo punto, nel caso in cui la variabile BOOST_INCLUDE_DIRS non sia settata, bisogna eseguire il seguente comando:

```
$ BOOST_INCLUDE_DIRS=/path/to/boost/include make all
```

Ciò può essere verificato mediante

```
$ echo BOOST_INCLUDE_DIRS.
```

Per ottenere il percorso in cui si trova boost:

```
$ whereis boost
```

Se la compilazione dà errore su Python.h, può significare che dall'installazione di Python manchino i file sorgenti, quindi è necessario cercare il nome della libreria installata e completare l'installazione.

- *\$ sudo apt search libpython*
- *\$ sudo apt install libpython3.8-dev*

Dopodiché ripetere

```
$ BOOST_INCLUDE_DIRS=/path/to/boost/include make all
```

A questo punto, l'installazione del pacchetto psbody-mesh è completata e la si può testare mediante *\$ make tests*.

Per ottenere la documentazione e visualizzarla:

- *\$ make documentation*
- *\$ firefox /path/to/mesh/doc/build/html/index.html*

7.0.2 Kaolin

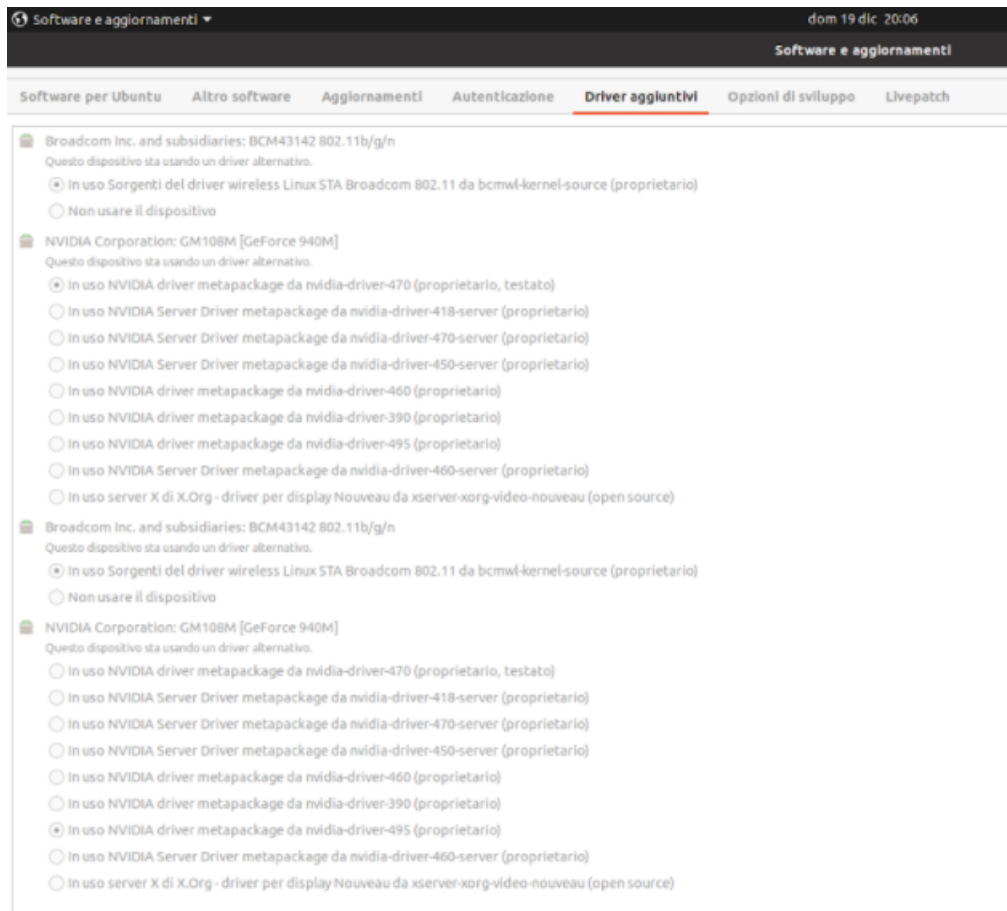
La libreria di NVIDIA Kaolin fa parte di una suite di strumenti per la ricerca sul deep learning 3D. Fornisce una API di PyTorch utile per lavorare con una varietà di rappresentazioni 3D, comprende una raccolta sempre crescente di operazioni ottimizzate per la GPU come rendering differenziabile modulare, conversioni veloci tra rappresentazioni, caricamento dei dati, checkpoint 3D e altro.

Requisiti:

- *Python* ≥ 3.6
- *CUDA* ≥ 10.0 (con *nvcc* installato)
- *torch* ≥ 1.5 , $\leq 1.7.1$
- *cython* $= 0.29.20$
- *scipy* $\geq 1.2.0$
- *Pillow* $\geq 8.0.0$
- *usd-core* ≥ 20.11 (opzionale, richiesto per le librerie di Python USD (Universal Scene Description)).

7.0.3 CUDA

```
$ sudo apt install nvidia-cuda-toolkit
```



```
$ sudo apt install cuda
```

Per assicurarsi di utilizzare la scheda grafica NVIDIA:

```
$ sudo prime-select intel
```

```
$ sudo prime-select nvidia
```

Per avere informazioni sulla scheda grafica:

```
$ nvidia-smi
```

7.0.4 Installazione di Kaolin

Per installare Kaolin¹¹:

- `$ git clone --recursive`
- `$ cd kaolin`
- `$ git checkout v0.9.0`
- `$ python setup.py develop`

¹¹<https://github.com/NVIDIAGameWorks/kaolin>

Se durante la fase di setup non dovessero essere trovati torch e/o cython, installarli utilizzando i seguenti comandi:

- `$ pip install torch torchvision`
- `$ pip install Cython`

nvcc warning : The 'compute_35', 'compute_37', 'compute_50', 'sm_35', 'sm_37' and 'sm_50' architectures are deprecated, and may be removed in a future release (Use -Wno-deprecated-gpu-targets to suppress warning).

Questo significa che, per la versione di kaolin 0.9, la GPU risulta deprecata.

L'installazione di kaolin v0.1, invece, dà errore perché manca la libreria pptk, la cui wheel non è più reperibile e anche la compilazione dai file sorgenti causa problemi.

Kaolin vuole gcc e g++ 8 e vuole che la versione di CUDA sia uguale a quella usata da Pytorch, che nell'ultima versione corrisponde a CUDA 10.2. CUDA 10.2 richiede gcc e g++ 9.

Bisogna trovare una versione di PyTorch che usi CUDA 9 (PyTorch 1.2 dovrebbe essere compatibile con CUDA 9), potrebbero tuttavia esserci delle interferenze di compatibilità con gcc v8.

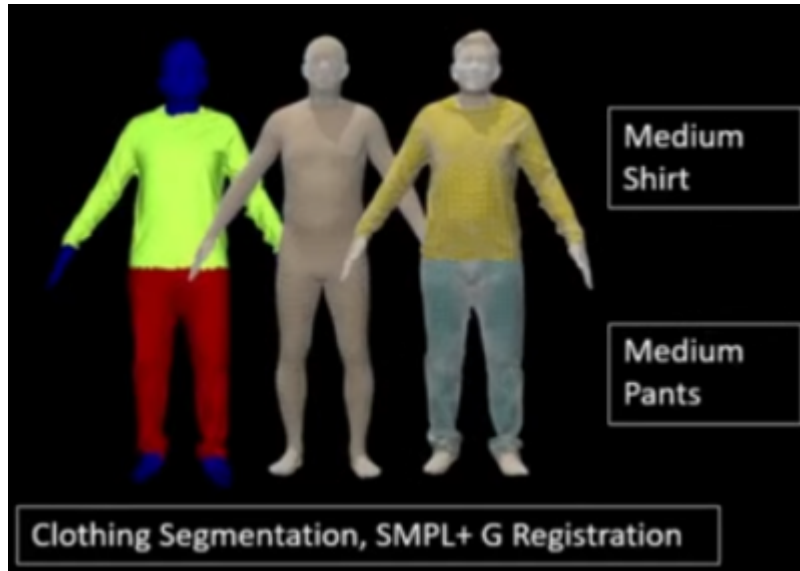
Per verificare la possibilità di importare kaolin, si può testare la sua installazione nel seguente modo:

```
$ python -c "import kaolin; print(kaolin.__version__)"
```

7.1 Download del dataset

```
$ git clone --recursive https://github.com/garvita-tiwari/sizer
```

Poiché il dataset SIZER include registrazioni sul modello SMPL, bisogna scaricare il modello SMPL¹² e salvarlo nella cartella di sizer training_data.



Settare

```
DATA_DIR='<downloaded dataset path>/training_data' in utils/global_var.py
```

A questo punto è possibile fare la validazione di ParserNet:

```
$ python trainer/parsernet_eval.py --log_dir <log_dir_path>
```

Si consiglia di salvare i file di log in una cartella dedicata all'interno di quella del progetto Sizer, invece che in trainer, in modo da non contaminare la cartella contenente i file di addestramento. Se i moduli tensorboardX e ipdb non dovessero essere trovati, procedere alla loro installazione mediante i seguenti comandi:

- `$ pip install tensorboardX`
- `$ pip install ipdb`

In caso di errori sugli import¹³¹⁴, come ad esempio `ImportError: cannot import name 'TriangleMesh' from 'kaolin.rep'`, si consiglia di effettuare il downgrade di torch alla versione 1.4, con la quale il codice sizer è stato testato, oppure, se il problema persiste, passare alla versione 0.1 di kaolin se non è già stato fatto, in quanto dalla versione 0.9 di Kaolin TriangleMesh non esiste più.

- `/kaolin$ git checkout v0.1`
- `/kaolin$ python setup.py develop`

¹²<https://smpl.is.tue.mpg.de/>

¹³<https://githubmemory.com/repo/NVIDIAGameWorks/kaolin/issues/414>

¹⁴<https://github.com/pytorch/pytorch/issues/38090>

8 Windows

Create a dedicated Python virtual environment and activate it:

```
> conda create -n my_env (garments)
```

```
> conda activate my_env
```

Per prima cosa bisogna installare le librerie di Boost. Si può decidere di seguire le linee guida nel file README¹⁵ oppure installarlo da github¹⁶.

Unzip il file boost e seguire le istruzioni presenti in index.html.

8.1 Boost

Leggere il file “Install” per maggiori informazioni.

- Andare nella directory tools -build-.
- Eseguire bootstrap.bat (sul prompt dei comandi scrivere:
`start bootstrap.bat`)
- Eseguire `b2install -prefix=PREFIX` dove PREFIX è la directory dove si vuole installare Boost.Build
`start b2 install --prefix=.-build`

Cerca il toolset adatto al compilatore che vuoi utilizzare su Pycharm dalla tabella nel seguente link: (una lista aggiornata è disponibile anche nella documentazione di *Boost.Build*)

https://www.boost.org/doc/libs/1_77_0/tools/build/doc/html/index.html#bbv2.reference.tools

8.2 Librerie Boost per GCC

Guida per installare le librerie di Boost per GCC (MinGW):

<https://gist.github.com/sim642/29caef3cc8afaa273ce6>

8.3 Psbody-mesh

A questo punto, bisogna compilare ed installare il pacchetto Psbody-mesh usando il Makefile:

- `cd .path/to/mesh-master`
- `make all`

In caso di errori quali

```
pip uninstall spyder
```

```
pip 10 no longer uninstalls distutils packages
```

fare il downgrade a pip 8.1.1.:

```
sudo -H pip3 install pip==8.1.1
```

Psbody-mesh contiene le principali funzioni di manipolazione e visualizzazione di mesh. Richiede Python3.5+ e le librerie di Boost installate seguendo il paragrafo precedente.

¹⁵https://www.boost.org/users/history/version_1_77_0.html

¹⁶<https://gist.github.com/sim642/29caef3cc8afaa273ce6>

8.4 Kaolin

E' necessaria l'installazione della libreria Kaolin, che può essere effettuata seguendo le linee guida del sito ufficiale:

<https://kaolin.readthedocs.io/en/latest/notes/installation.html>

La libreria di NVIDIA Kaolin fa parte di una suite di strumenti per la ricerca sul deep learning 3D. Fornisce una API di PyTorch utile per lavorare con una varietà di rappresentazioni 3D, comprende una raccolta sempre crescente di operazioni ottimizzate per la GPU come rendering differenziabile modulare, conversioni veloci tra rappresentazioni, caricamento dei dati, checkpoint 3D e altro.

Requisiti:

- *Python* ≥ 3.6
- *CUDA* ≥ 10.0 (con *nvcc* installato)
- *torch* ≥ 1.5 , $\leq 1.7.1$
- *cython* $= 0.29.20$
- *scipy* $\geq 1.2.0$
- *Pillow* $\geq 8.0.0$
- *usd-core* ≥ 20.11 (opzionale, richiesto per le librerie di *Python USD*) (*Universal Scene Description*).

È possibile eseguire il file di setup di kaolin tramite il terminale di pycharm, all'interno del progetto: `python setup.py install`

8.5 Sizer dataset

Scaricare il dataset dalla sezione Data¹⁷ della pagina web di SIZER, recuperarne il percorso e seguire le istruzioni del README¹⁸ di github.

8.6 Installazione dei pacchetti

Per prima cosa è necessario assicurarsi che il pacchetto pip per python 3 sia installato correttamente.

In caso contrario, sarà sufficiente eseguire la seguente linea di codice sul terminare direttamente offertoci da pycharm

```
py -m ensurepip --default-pip
```

I pacchetti richiesti dal modello sono 131, facilmente reperibili dal file requirements.txt presente all'interno del github¹⁹ di SIZER.

Per scaricare le librerie richieste dal file .txt è sufficiente spostarsi, tramite l'utilizzo della shell di python, nella cartella contenente il file requirements.txt ed eseguire il seguente comando:

¹⁷<http://virtualhumans.mpi-inf.mpg.de/sizer/>

¹⁸<https://github.com/garvita-tiwari/sizer>

¹⁹<https://github.com/garvita-tiwari/sizer/blob/master/requirements.txt>


```
pip install -r requirements.txt
```

Tuttavia il nome "completo" dei pacchetti inseriti nel documento di testo requirements.txt non corrisponde sempre alla versione richiesta da SIZER. Per ovviare a questo problema è necessario installare manualmente ogni singolo pacchetto.

La maggior parte dei pacchetti è facilmente installabile su pycharm eseguendo il comando pip install [nome pacchetto]

Tra questi pacchetti, alcuni necessitano il supporto del terminale di anaconda per essere installati. Di seguito vengono specificati più comandi d'installazione per ogni singolo pacchetto in quanto diverse versioni di sistema operativo richiedono diversi comandi compatibili con i rispettivi registri.

Per quanto riguarda i seguenti pacchetti è necessariamente richiesto Linux come sistema operativo.

Alcuni di questi pacchetti sono necessari per abilitare la fase di training su Pycharm.

References

- [1] Hugo Bertiche, Meysam Madadi, Emilio Tylson, and Sergio Escalera. Deepsd: Automatic deep skinning and pose space deformation for 3d garment animation. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 5471–5480, 2021.
- [2] Bharat Lal Bhatnagar, Garvita Tiwari, Christian Theobalt, and Gerard Pons-Moll. Multi-garment net learning to dress 3d people from images. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 5420–5430, 2019.
- [3] Valentin Gabeur, Jean-Sébastien Franco, Xavier Martin, Cordelia Schmid, and Gregory Rogez. Moulding humans non-parametric 3d human shape estimation from single images. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 2232–2241, 2019.
- [4] Nils Hasler, Carsten Stoll, Martin Sunkel, Bodo Rosenhahn, and H-P Seidel. A statistical model of human pose and body shape. In *Computer graphics forum*, volume 28, pages 337–346. Wiley Online Library, 2009.
- [5] Angjoo Kanazawa, Michael J Black, David W Jacobs, and Jitendra Malik. End-to-end recovery of human shape and pose. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7122–7131, 2018.
- [6] Christoph Lassner, Javier Romero, Martin Kiefel, Federica Bogo, Michael J Black, and Peter V Gehler. Unite the people closing the loop between 3d and 2d human representations. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 6050–6059, 2017.
- [7] Matthew Loper, Naureen Mahmood, Javier Romero, Gerard Pons-Moll, and Michael J Black. Smpl a skinned multi-person linear model. In *Seminal Graphics Papers Pushing the Boundaries, Volume 2*, pages 851–866. 2023.
- [8] Alexandros Neophytou and Adrian Hilton. A layered model of human body and garment deformation. In *2014 2nd International Conference on 3D Vision*, volume 1, pages 171–178. IEEE, 2014.
- [9] Gerard Pons-Moll, Sergi Pujades, Sonny Hu, and Michael J Black. Clothcap: Seamless 4d clothing capture and retargeting. *ACM Transactions on Graphics (ToG)*, 36(4):1–15, 2017.
- [10] Soubhik Sanyal, Alex Vorobiov, Timo Bolkart, Matthew Loper, Betty Mohler, Larry S Davis, Javier Romero, and Michael J Black. Learning realistic human posing using cyclic self-supervision with 3d shape, pose, and appearance consistency. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 11138–11147, 2021.
- [11] Garvita Tiwari, Bharat Lal Bhatnagar, Tony Tung, and Gerard Pons-Moll. Sizer: A dataset and model for parsing 3d clothing and learning size sensitive 3d clothing. In *European Conference on Computer Vision*, pages 1–18. Springer, 2020.

Package List (Windows or Linux)

Nome e versione del pacchetto

libwebp==1.2.0=h89dd481_0
 sqlite==3.36.0=hc218d9a_0
 sqlite==3.36.0=hc218d9a_0
 libwebp-base==1.2.0=h27cfd23_0
 xz==5.2.5=h7b6447c_0
 yaml==0.2.5=h516909a_0
 pexpect==4.8.0=pypi_0
 jpeg==9d=h7f8727e_0
 numpy==1.21.2=py38h20f2e39_0
 giflib==5.2.1=h7b6447c_0
 libuv==1.40.0=h7b6447c_0
 libtiff==4.2.0=h85742a9_0
 mkl_fft==1.3.1=py38hd3c417c_0
 lcms2==2.12=h3be6417_0
 mkl_random==1.2.2=py38h51133e4_0
 mkl-service==2.4.0=py38h7f8727e_0
 cudatoolkit==10.2.89=hfd86e86_1
 scikit-image==0.18.3=pypi_0
 tk==8.6.11=h1ccaba5_0
 numpy-base==1.21.2=py38h79a1101_0
 lz4-c==1.9.3=h295c915_1
 zstd==1.4.9=haebb681_0
 libpng==1.6.37=hbc83047_0
 openssl==1.1.1l=h7f8727e_0
 torchvision==0.8.2=cpu_py38ha229d99_0
 cachetools==4.2.4=pypi_0
 pytorch==1.7.1=py3.8_cuda10.2.89_cudnn7.6.5_0
 libffi==3.3=he6710b0_2

liblapack==3.9.0=12_linux64_mkl

freetype==2.11.0=h70c0345_0

Shapely-1.8.0

igl==2.2.1=py38h52fb889_1

ca-certificates==2021.10.8=ha878542_0

Comandi per anaconda

conda install libwebp==1.2.0
 conda install sqlite==3.36.0
 conda install sqlite==3.36.0
 conda install libwebp-base==1.2.0
 conda install xz==5.2.5
 conda install yaml==0.2.5
 conda install pexpect==4.8.0
 conda install jpeg==9d
 conda install numpy==1.21.2
 conda install giflib==5.2.1
 conda install libuv==1.40.0
 conda install libtiff==4.2.0
 conda install mkl_fft==1.3.1
 conda install lcms2==2.12
 conda install mkl_random==1.2.2
 conda install mkl-service==2.4.0
 conda install cudatoolkit==10.2.89
 conda install scikit-image==0.18.3
 conda install tk==8.6.11
 conda install numpy-base==1.21.2
 conda install lz4-c==1.9.3
 conda install zstd==1.4.9
 conda install libpng==1.6.37
 conda install openssl==1.1.1l
 conda install -c pytorch-lts torchvision
 conda install -c quantopian cachetools
 conda install -c fastchan pytorch
 conda install -c conda-forge libffi
 conda install -c conda-forge/label/gcc7 libffi
 conda install -c conda-forge/label/broken libffi
 conda install -c conda-forge/label/cf201901 libffi
 conda install -c conda-forge/label/cf202003 libffi
 conda install -c conda-forge liblapack
 conda install -c conda-forge/label/broken liblapack
 conda install -c conda-forge/label/cf202003 liblapack
 conda install -c conda-forge freetype
 conda install -c conda-forge/label/gcc7 freetype
 conda install -c conda-forge/label/cf201901 freetype
 conda install -c conda-forge/label/cf202003 freetype
 conda install -c conda-forge shapely
 conda install -c conda-forge/label/gcc7 shapely
 conda install -c conda-forge/label/cf201901 shapely
 conda install -c conda-forge/label/cf202003 shapely
 conda install -c conda-forge/label/shapely_dev shapely
 conda install -c conda-forge igl
 conda install -c conda-forge/label/cf202003 igl
 conda install -c conda-forge ca-certificates
 conda install -c conda-forge/label/gcc7 ca-certificates
 conda install -c conda-

Package List (Linux only)

Nome e versione del pacchetto	comandi per anaconda
libgomp==9.3.0=h5101ec6_17	conda install -c conda-forge libgomp conda install -c conda-forge/label/broken libgomp conda install -c conda-forge/label/cf202003 libgomp
libstdcxx-ng==9.3.0=hd4cf53a_17	conda install -c conda-forge libstdcxx-ng conda install -c conda-forge/label/gcc7 libstdcxx-ng conda install -c conda-forge/label/broken libstdcxx-ng conda install -c conda-forge/label/cf201901 libstdcxx-ng conda install -c conda-forge/label/cf202003 libstdcxx-ng
ld_impl_linux-64==2.35.1=h7274673_9	conda install -c conda-forge ld_impl_linux-64 conda install -c conda-forge/label/broken ld_impl_linux-64 conda install -c conda-forge/label/cf202003 ld_impl_linux-64
_openmp_mutex==4.5=1_gnu	conda install -c conda-forge _openmp_mutex conda install -c conda-forge/label/cf202003 _openmp_mutex
libgfortran5==11.2.0=h5c6108e_11	conda install -c conda-forge libgfortran5 conda install -c conda-forge/label/broken libgfortran5
libblas==3.9.0=12_linux64_mkl	conda install -c nogil libblas conda install -c cctbx202112 libblas
libgfortran-ng==11.2.0=h69a702a_11	conda install -c conda-forge libgfortran-ng conda install -c conda-forge/label/broken libgfortran-ng conda install -c conda-forge/label/cf201901 libgfortran-ng conda install -c conda-forge/label/cf202003 libgfortran-ng
ncurses==6.3=h7f8727e_2	conda install -c conda-forge ncurses conda install -c conda-forge/label/gcc7 ncurses conda install -c conda-forge/label/cf201901 ncurses conda install -c conda-forge/label/cf202003 ncurses
readline==8.1=h27cfd23_0	conda install -c conda-forge readline conda install -c conda-forge/label/gcc7 readline conda install -c conda-forge/label/cf201901 readline conda install -c conda-forge/label/cf202003 readline
nvidiacub==1.10.0=0	conda install -c bottler nvidiacub
libgcc-ng==9.3.0=h5101ec6_17	conda install -c conda-forge libgcc-ng conda install -c conda-forge/label/gcc7 libgcc-ng conda install -c conda-forge/label/broken libgcc-ng conda install -c conda-forge/label/cf201901 libgcc-ng

- [12] Gul Varol, Duygu Ceylan, Bryan Russell, Jimei Yang, Ersin Yumer, Ivan Laptev, and Cordelia Schmid. Bodynet volumetric inference of 3d human body shapes. In *Proceedings of the European conference on computer vision (ECCV)*, pages 20–36, 2018.
- [13] Gul Varol, Javier Romero, Xavier Martin, Naureen Mahmood, Michael J Black, Ivan Laptev, and Cordelia Schmid. Learning from synthetic humans. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 109–117, 2017.
- [14] Kota Yamaguchi, M Hadi Kiapour, Luis E Ortiz, and Tamara L Berg. Parsing clothing in fashion photographs. In *2012 IEEE Conference on Computer vision and pattern recognition*, pages 3570–3577. IEEE, 2012.
- [15] Chao Zhang, Sergi Pujades, Michael J Black, and Gerard Pons-Moll. Detailed, accurate, human shape estimation from clothed 3d scan sequences. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4191–4200, 2017.